

Control Files to a ParaDiS Simulation

Source-level parsing, validation, and runtime propagation

ParaDiS development notes

August 2026

Abstract

This guide follows one control-file value from text on disk to the code that consumes it during a ParaDiS run. It identifies the files involved, reproduces the relevant source with repository line numbers, and explains each line or small group of lines. The emphasis is on the actual implementation in this repository: the control-file grammar, the `Param_t` storage object, command-line precedence, rank-0 parsing, MPI propagation, data/restart headers, derived defaults, and the first-cycle consumers.

Contents

1	Executive view	3
2	Files and responsibilities	3
3	What counts as input?	4
4	Storage: how a name reaches <code>Param_t</code>	4
4.1	Registry metadata	4
5	Program entry and input-file selection	6
5.1	Main loop	6
5.2	MPI and the initialization call chain	7
6	Initialization: defaults, CLI, and the control file	9
7	Building the schema: <code>CtrlParamInit</code>	15
7.1	The binding operation	20
8	Lexing and parsing, line by line	21
8.1	<code>ReadControlFile</code>	21
8.2	Tokenization	22
8.3	Lookup and conversion	25
9	Precedence and MPI propagation	29
10	Validation and derived values	30
11	The nodal data/restart boundary	37
12	From <code>Param_t</code> to the first simulation cycle	41
12.1	Cycle count and integrator	41
12.2	Topology, remeshing, and cross slip	42

13 A concrete value trace	46
14 Restart persistence and debugging	46
15 Adding a new control parameter	48
16 Implementation caveats	49
17 Source index	49

1 Executive view

The complete path is:

```
argv / control.script
      | ParaDiS.cc -> ParadisInit.cc -> Initialize.cc
      v
CtrlParamInit: names, types, destinations, defaults
      |
ReadControlFile -> GetNextToken -> LookupParam -> GetParamVals
      |
command-line overrides -> InputSanity -> material/table setup
      |
MPI_Bcast(Param_t) to every rank
      |
nodal data/restart header -> geometry and node state
      |
SetRemainingDefaults + SetMaterialTypeFromMobility
      |
ParadisStep: forces, integrator, topology, remeshing, output
```

Only the registered entries in `CtrlParamInit()` are legal run-control keys. A field in `Param_t` is not automatically readable merely because it exists in the structure.

2 Files and responsibilities

File	Responsibility
src/ParaDiS.cc	Process entry point; calls initialization and runs the cycle loop.
src/ParadisInit.cc,	Allocate the <code>Home_t</code> object, initialize MPI/rank information, then
src/InitHome.cc	call <code>Initialize</code> .
src/Initialize.cc	Select input files, allocate parameter lists, parse the control file, apply CLI precedence, validate, broadcast, read nodal data, and derive runtime values.
src/Param.cc	Registers control and data names with <code>BindVar</code> ; assigns defaults.
src/Parse.cc, include/	Tokenizer, lookup, type conversion, list parsing, aliases, and restart-
Parse.h	file writing.
src/ReadRestart.cc	Reads the text control file and text nodal-data/restart headers.
src/ReadBinaryRestart.cc	Reads the binary restart/data representation.
src/InputSanity.cc	Cross-field validation and legal-value normalization.
include/Param.h, include/	Define storage, registry metadata, and the pointer through which
Typedefs.h, include/Home.	simulation code accesses parameters.
h	
src/ElasticConstants.cc,	Consume filenames and material controls after parsing.
mobility files	
src/ParadisStep.cc,	First-cycle consumers of the propagated values.
src/Topology.cc,	
src/Remesh.cc, src/	
CrossSlip.cc	

3 What counts as input?

ParaDiS has four related but distinct input surfaces:

1. The run control file (normally `control.script`; a positional argument or `-f` can select another file).
2. Command-line switches such as `-maxstep`, `-doms`, `-cells`, `-dir`, and `-r`. These are applied after the text file and therefore have higher precedence.
3. A nodal data or restart file. Its header is parsed by the separate `DataParamList`; it is not another control file.
4. External tables named by control strings, for example `ecFile`, `meltTempFile`, `MobFileNL`, and correction-table filenames.

For example, `tests/bcc_binary_junction_node.ctrl` contains scalar assignments, quoted strings, and bracketed vectors. Representative lines are shown below (the source file itself is the authoritative test fixture):

```
dirname = "./data"
numXdoms = 1
maxstep = 100
mobilityLaw = "BCC_0"
appliedStress = [ 0 0 0 0 0 0 ]
edotdir = [ 1 0 0 ]
```

The parser treats `appliedStress=[...]` as six values and `edotdir=[...]` as three values; the registration determines the required count.

4 Storage: how a name reaches Param_t

4.1 Registry metadata

```
97  /*
98   *      Define a couple structures used for managing the control
99   *      and data file parameter lists
100  */
101  typedef struct {
102      char varName[MAX_STRING_LEN];
103      int  valType;
104      int  valCnt;
105      int  flags;
106      void *vallist;
107  } VarData_t;
108
109
110  typedef struct {
111      int      paramCnt;
112      VarData_t *varList;
113  } ParamList_t;
```

97–101 The registry stores a variable name and the address of its destination. The address is a `void*` because values may be integer, double, or character data.

102–107 `valType` says how tokens are converted; `valCnt` is the scalar/vector cardinality; `flags` records alias, disabled, and user-set state.

108–113 `ParamList_t` is a growable array of these records. There is no map or copied value: parsing writes directly through `vallist` into `Param_t`.

```

261     int         cycle           ;  < current cycle
262     int         lastCycle      ;  < cycle to quit on
263
264     long long    utc_cycle_start ;  < wall clock time at start of simulation (utc microsecs)
265     long long    utc_cycle_prev ;  < wall clock time of previous cycle      (utc microsecs)
266     long long    utc_cycle_curr ;  < wall clock time of current cycle      (utc microsecs)
267
268     Param_t      *param;
269
270     /*
271      * Transient state used while the drift-mode subcycling integrator
272      * is active. The state is allocated and released within a single
273      * outer ParaDiS cycle, before any topology changes can invalidate
274      * its segment-pair keys.
275      */
276     Subcycling_t *subcycling;
277
278 #ifdef ANISOTROPIC
279     AnisotropicVars_t anisoVars;
280 #ifdef TAYLORFMM
281     FManisoTable_t    *fmAnisoTable;
282 #endif
283 #endif
284
285 #ifdef SPECTRAL
286     Spectral_t *spectral;
287 #endif
288
289 /*
290  * The following two pointers are used for registering control
291  * and data file parameters and when reading/writing restart
292  * files. They will only be set and used in task zero.
293  */
294     ParamList_t *ctrlParamList;
295     ParamList_t *dataParamList;

```

The important lines are the `Param_t*param` member and the two root-only lists. Rank zero owns the names and pointers; all ranks receive the plain parameter bytes later.

The token and value constants are defined here:

```

14 // Define a set values that may be returned by
15 // functions used in parsing the user supplied
16 // values in the control file.
17
18 #define TOKEN_ERR      -1
19 #define TOKEN_NULL     0
20 #define TOKEN_GENERIC  1
21 #define TOKEN_EQUAL    2
22 #define TOKEN_BEGIN_VAL_LIST  3
23 #define TOKEN_END_VAL_LIST   4
24
25 // Define the variable types that may be associated with input
26 // parameters
27
28 #define V_NULL    0
29 #define V_DBL    1
30 #define V_INT    2
31 #define V_STRING 3
32 #define V_COMMENT 4
33

```

```

34 #define VFLAG_NULL      0x00
35 #define VFLAG_ALIAS     0x01
36 #define VFLAG_INITIALIZED 0x02
37 #define VFLAG_DISABLED  0x04
38 #define VFLAG_SET_BY_USER 0x08
39
40 //
-----
41
42 extern void BindVar          (ParamList_t *list, const char *name, const void *addr, const int
      type, const int cnt, const int flags);
43 extern void DisableUnneededParams (Home_t      *home);
44 extern int  GetNextToken      (FILE           *fd , char *token, int maxTokenSize);
45 extern int  GetParamVals     (FILE           *fd , int  vType, int vExpected, void *vList);
46 extern int  LookupParam      (ParamList_t *list, const char *token);
47 extern void MarkParamDisabled (ParamList_t *list, const char *name );
48 extern void MarkParamEnabled  (ParamList_t *list, const char *name );
49 extern void WriteParam        (ParamList_t *list, const int  index, FILE *fd);

```

TOKEN_ERR, TOKEN_NULL, TOKEN_GENERIC, and the bracket/equal tokens describe the grammar. V_DBL, V_INT, and V_STRING select conversion. VFLAG_SET_BY_USER is set only after a key is actually read.

5 Program entry and input-file selection

5.1 Main loop

```

166 /*
167  *      Now do the basic initialization (i.e. init MPI, read restart
168  *      files, blah, blah, blah...)
169  */
170      ParadisInit(&home,argv,argv);
171
172 /*
173  * Determine the critical stress of a FR source
174  */
175
176 #ifdef FRCRIT
177     if (home->param->fmEnabled) Fatal("Critical stress calcs not implemented for FMM yet.\n");
178     Calc_FR_Critical_Stress(argv, argv, &home);
179     ParadisFinish(home);
180     exit(0);
181 #endif
182
183     home->cycle      = home->param->cycleStart;
184
185     param            = home->param;
186     cycleEnd         = param->cycleStart + param->maxstep;
187     initialDLBCycles = param->numDLBCycles;

```

Lines 166–167 call initialization. Lines 169–172 copy the start cycle and compute `cycleEnd=cycleStart+maxstep`; this is the first direct use of a parsed control value. Lines 179–187 establish the initial time/cycle state.

```

245     while (home->cycle < cycleEnd)
246     {
247
248 #ifdef USE_CALIPER

```

```

249         CALI_CXX_MARK_LOOP_ITERATION(cycle, home->cycle);
250     #endif
251
252     ParadisStep(home);
253
254     TimerClearAll(home);
255
256     #ifndef SHUTDOWN_SIGNAL
257         if (DoForcedShutdown(home)) {
258             break;
259         }
260     #endif
261
262     /*
263     *     If the user specified a simulation time at which to terminate
264     *     the simulation and we've reached/exceeded that time, then
265     *     terminate the simulation even if we have not run for the
266     *     specified number of cycles.
267     */
268     if ((param->timeEnd > 0.0) && (param->timeNow >= param->timeEnd)) {
269         if (home->myDomain == 0) {
270             printf("+++\\n"
271                 "+++ Requested simulation end time: %.10e seconds\\n"
272                 "+++ Current simulation time:         %.10e seconds\\n"
273                 "+++\\n"
274                 "+++ Terminating simulation now...\\n"
275                 "+++\\n", param->timeEnd, param->timeNow);
276         }
277
278         break;
279     }
280 }

```

The loop repeatedly calls `ParadisStep`; the time-end test uses the initialized `timeNow` and `timeEnd`. Thus a control value can affect both the number of loop iterations and the termination condition.

5.2 MPI and the initialization call chain

```

247 void ParadisInit
248 (
249     Home_t **homeptr ,    ///< address of pointer to Home data structure (allocate/initialized here)
250     int      argc      ,  ///< application argument count
251     char    *argv[]     ///< application argument vector
252 )
253 {
254
255     Home_t *home      = InitHome();
256     *homeptr = home;
257
258     TimerInit(home);
259
260     #ifndef PARALLEL
261     /*
262     *     If ParaDiS has been compiled with OpenMP, initialize the MPI
263     *     library for threads, otherwise do standard MPI initialization
264     *
265     *     NOTE: The standard ParaDiS build only requires MPI thread support
266     *           at the MPI_THREAD_FUNNELED level, but when compiled with
267     *           the hooks for communicating with the ARL FEM code (fed3)

```

```

268 *           the required MPI thread support level is MPI_THREAD_MULTIPLE.
269 */
270 #ifndef _OPENMP
271
272 #ifdef _ARLFEFEM
273     int requiredLevel = MPI_THREAD_MULTIPLE;
274 #else
275     int requiredLevel = MPI_THREAD_FUNNELED;
276 #endif /* ifdef _ARLFEFEM */
277
278     int providedLevel = MPI_THREAD_SINGLE;           // (default)
279     MPI_Init_thread(&argc, &argv, requiredLevel, &providedLevel); // (sets actual provided level
280 )
281 #else /* _OPENMP not defined */
282     MPI_Init(&argc, &argv);
283
284 #endif /* end ifdef _OPENMP */
285
286 #ifdef USE_SCR
287 /*
288  *       Initialize the Scalable Checkpoint/Restart library. Must be done
289  *       *after* MPI_Init() is called.
290  */
291     SCR_Init();
292 #endif
293
294     MPI_Comm_rank(MPI_COMM_WORLD, &home->myDomain );
295     MPI_Comm_size(MPI_COMM_WORLD, &home->numDomains );
296
297     MPI_Comm_rank(MPI_COMM_WORLD, &home->mpi_rank );
298     MPI_Comm_size(MPI_COMM_WORLD, &home->mpi_size );
299     MPI_Node_Rank(home->mpi_node_rank, home->mpi_node_size);
300     MPI_Hostname (home->mpi_hostname, sizeof(home->mpi_hostname));
301 #endif // ifdef PARALLEL
302
303 #ifndef PARALLEL
304     // defaults when MPI is not active...
305
306     home->myDomain      = 0;
307     home->numDomains    = 1;
308
309     home->mpi_rank      = 0;
310     home->mpi_size      = 1;
311     home->mpi_node_rank = 0;
312     home->mpi_node_size = 1;
313     gethostname(home->mpi_hostname, sizeof(home->mpi_hostname));
314
315 #endif // ifndef PARALLEL
316
317 #if defined PARALLEL & defined _OPENMP
318 /*
319  *       If MPI and threads are both enabled, make sure the MPI library
320  *       provides a sufficient level of threading support.
321  */
322     if (providedLevel != requiredLevel)
323     {
324         char *levelStr=0;
325
326         switch (providedLevel)
327         {

```



```

328         case (MPI_THREAD_SINGLE      ) : { levelStr = (char *) "MPI_THREAD_SINGLE"      ; break;
329     }
330         case (MPI_THREAD_FUNNELED     ) : { levelStr = (char *) "MPI_THREAD_FUNNELED"     ; break;
331     }
332         case (MPI_THREAD_SERIALIZED) : { levelStr = (char *) "MPI_THREAD_SERIALIZED"; break;
333     }
334         case (MPI_THREAD_MULTIPLE    ) : { levelStr = (char *) "MPI_THREAD_MULTIPLE"    ; break;
335     }
336         default
337             : { levelStr = (char *) "UNKNOWN"
338             ; break;
339     }
340     }
341     if (home->myDomain == 0)
342         Fatal("ParaDiS requires MPI thread support level MPI_THREAD_FUNNELED \n"
343             "but the current MPI version only provides level %s.", levelStr);
344 }
345 #endif /* end of 'if defined PARALLEL & defined _OPENMP' */
346
347 TimerStart(home,TOTAL_TIME);
348
349 TimerStart(home,INITIALIZE);
350 Initialize(home,argc,argv );
351 TimerStop (home,INITIALIZE);
352
353 /*
354  * For parallel runs with shared memory support enabled, we need
355  * to identify MPI tasks co-located on compute-nodes and set up
356  * MPI communicators for collective communications specific to
357  * those tasks.
358  */
359 GetNodeTaskGroups(home);
360
361 #ifdef _ARLFEM
362 FEM_Init(home);
363 #endif
364
365 #ifdef PARALLEL
366 MPI_Barrier(MPI_COMM_WORLD);
367 #endif
368
369 ParadisInit_Summary(home);
370
371 if (home && (home->myDomain==0) )
372     printf("ParadisInit finished\n");
373
374 return;

```

Lines 255–258 create/initialize `Home_t`. Lines 294–300 query MPI rank and size; the serial branch (303–315) supplies rank zero and one process. Lines 342–346 call `Initialize`, and lines 364–369 print the resulting setup summary. `Initialize` is therefore the boundary where textual input becomes simulation state.

6 Initialization: defaults, CLI, and the control file

```

1498 void Initialize
1499 (
1500     Home_t *home,    ///< pointer to initialized home structure

```

```

1501     int    argc,      ///< application argument count
1502     char   *argv[]    ///< application argument vector
1503 )
1504 {
1505     char      tmpDataFile[256], testFile[256];
1506
1507     char      *ctrlFile=0;
1508     char      *dataFile=0;
1509     InData_t   *inData  =0;
1510
1511     // Allocate some buffers used for MPI reduction operations in
1512     // various portions of ParaDiS. NOTE: most of the code that
1513     // uses these buffers need buffers of <numDomains> elements, but
1514     // at least 1 portion requires <numDomains>+1 elements, so
1515     // we allocate for the maximum size.
1516
1517     home->localReduceBuf = (int *)calloc(1, (home->numDomains+1) * sizeof(int));
1518     home->globalReduceBuf = (int *)calloc(1, (home->numDomains+1) * sizeof(int));
1519
1520     // Initialize map between old and new node tags before
1521     // reading nodal data and distributing it to the remote
1522     // domains.
1523
1524     home->tagMap      = (TagMap_t *)NULL;
1525     home->tagMapSize = 0;
1526     home->tagMapEnts = 0;
1527
1528     // Init defaults for some command-line parameters...
1529
1530     int    skipIO=0;                // skip IO reading
1531     int    numDLBCycles=0;          // initial dynamic load balance cycles
1532     int    maxNumThreads=1;         // OpenMP thread max
1533     int    ndx=0, ndy=0, ndz=0;     // domain geometry
1534     int    ncx=0, ncy=0, ncz=0;     // cell geometry
1535     int    doBinRead=0;             // disable binary read
1536     int    gpu_enabled=0;           // disble gpu
1537     int    maxstep=0;               // max number of cycles
1538     char   *dirname=0;              // output directory
1539
1540     Param_t *param=0;
1541
1542     if (home->myDomain != 0) {
1543         param = home->param = (Param_t *)calloc(1, sizeof(Param_t));
1544     }
1545
1546     inData = (InData_t *) calloc(1, sizeof(InData_t));
1547
1548     #ifdef _OPENMP

```

Lines 1498–1503 enter initialization. Lines 1505–1540 set local command-line variables to safe sentinels. Non-root ranks allocate only Param_t (1542–1544); they do not parse text.

```

1578     if (home->myDomain == 0)
1579     {
1580         int ignoreNonLoopFlux = 0;
1581         char *fluxSelection = (char *)NULL;
1582
1583         dataFile = (char *) NULL;
1584         ctrlFile = (char *) "control.script";
1585

```

```

1586     for (int i=1; i<argc; i++)
1587     {
1588         if ( strcmp(argv[i], "-r")==0 )
1589         {
1590             if (i >= (argc - 1)) Usage(argv[0]);
1591
1592             numDLBCycles = atoi(argv[++i]);
1593
1594             if (numDLBCycles <= 0) Usage(argv[0]);
1595         }
1596         else if ( (strcmp(argv[i], "-n" )==0)
1597                 || (strcmp(argv[i], "-nthrd")==0) )
1598         {
1599             if (i >= (argc - 1)) Usage(argv[0]);
1600             maxNumThreads = atoi(argv[++i]);
1601 #ifndef _OPENMP
1602             printf("WARNING: Not compiled with OpenMP support; '-n' option ignored.\n");
1603 #else
1604             omp_set_num_threads(maxNumThreads);
1605             if ((maxNumThreads < 1) ||
1606                 (maxNumThreads != omp_get_max_threads()))
1607             {
1608                 Fatal("Error: unable to set thread count to %d!\n", maxNumThreads);
1609             }
1610 #endif
1611         }
1612         else if ( (strcmp(argv[i], "-maxstep" )==0)
1613                 || (strcmp(argv[i], "-nstep" )==0) )
1614         {
1615             if (i >= (argc - 1)) Usage(argv[0]);
1616             maxstep = atoi(argv[++i]);
1617         }
1618         else if (strcmp(argv[i], "-b")==0) { doBinRead = 1; }
1619         else if (strcmp(argv[i], "-s")==0) { skipIO = 1; }
1620         else if (strcmp(argv[i], "-d")==0)
1621         {
1622             if (i >= (argc - 1)) Usage(argv[0]);
1623             dataFile = (char *) argv[++i];
1624
1625             // This argument is provided for some site-specific
1626             // post-processing and as such is not otherwise
1627             // documented or supported. If you don't know what
1628             // it is for, there's no reason to use it.
1629
1630         }
1631         else if (strcmp(argv[i], "-f")==0)
1632         {
1633             if (i >= (argc - 1)) { Fatal("Missing argument for '-f' option"); }
1634             fluxSelection = (char *) argv[++i];
1635         }
1636         else if (strcmp(argv[i], "-doms")==0)
1637         {
1638             // '-doms' is used to override the domain specification in the control file
1639
1640             if (i >= (argc - 1)) { Fatal("Missing arguments for '-dom' option"); }
1641
1642             ndx = atoi(argv[++i]); // (number of domains (x))
1643             ndy = atoi(argv[++i]); // (number of domains (y))
1644             ndz = atoi(argv[++i]); // (number of domains (z))
1645         }
1646         else if (strcmp(argv[i], "-cells")==0)

```

```

1647     {
1648         // '-cells' is used to override the cell specification in the control file
1649
1650         if (i >= (argc - 1)) { Fatal("Missing arguments for '-cell' option"); }
1651
1652         ncx = atoi(argv[++i]); // (number of cells (x)
1653         ncy = atoi(argv[++i]); // (number of cells (y)
1654         ncz = atoi(argv[++i]); // (number of cells (z)
1655     }
1656     else if (strcmp(argv[i], "-dir")==0)
1657     {
1658         // '-dir' is used to override the save directory specification in the control
1659         file
1660
1661         if (i >= (argc - 1)) { Fatal("Missing arguments for '-dir' option"); }
1662
1663         dirname = (char *) argv[++i]; // path to destination directory
1664     }
1665     else if (strcmp(argv[i], "-gpu")==0)
1666     {
1667         #ifdef GPU_ENABLED
1668             gpu_enabled=1;
1669         #else
1670             printf("WARNING: Not compiled with GPU support; '-gpu' option ignored.\n");
1671             gpu_enabled=0;
1672         #endif
1673     }
1674     else
1675     {
1676         if (i < (argc - 1)) Usage(argv[0]);
1677         ctrlFile = (char *) argv[i];
1678     }
1679 }
1680
1681 // The fluxSelection is related to some site-specific
1682 // post-processing only and as such should never be set
1683 // in *normal* ParaDiS simulations
1684
1685 if (fluxSelection != (char *)NULL)
1686 {
1687     if (StrEquiv(fluxSelection, "loops")) { ignoreNonLoopFlux=1; }
1688     else { Fatal("Unrecognized argument for '-f' option"
1689 ); }
1690 }
1691
1692 // If the user did not specify a data file name, set
1693 // a default name based on the base control file name.
1694
1695 if (dataFile == (char *)NULL)
1696 {
1697     strcpy(tmpDataFile, ctrlFile);
1698
1699     char *start = strrchr(tmpDataFile, '/');
1700
1701     if (start == (char *)NULL) start = tmpDataFile;
1702
1703     char *sep = strrchr(start, '.');
1704
1705     if ((sep != (char *)NULL) && (sep > start)) *sep = 0;
1706
1707     // If the user specified that a binary data file is

```

```

1706         // used set default name accordingly. If the user
1707         // did not specify, try to figure out whether to
1708         // use a binary restart or not.
1709
1710         if (doBinRead)
1711         {
1712             strcat(tmpDataFile, HDF_DATA_FILE_SUFFIX); // append ".hdf"
1713         }
1714         else
1715         {
1716             strcpy(testFile, tmpDataFile);
1717             strcat(testFile, HDF_DATA_FILE_SUFFIX);      // append ".hdf"
1718
1719             int fd = open(testFile, O_RDONLY, 0);
1720
1721             if (fd<0)
1722             {
1723                 strcat(testFile, ".0");                // append ".0"
1724                 fd = open(testFile, O_RDONLY, 0);        // (try again)
1725             }
1726
1727             if (fd>=0)
1728             {
1729                 doBinRead=1;
1730                 close(fd);
1731                 strcat(tmpDataFile, HDF_DATA_FILE_SUFFIX); // append ".hdf"
1732             }
1733             else
1734             {
1735                 strcat(tmpDataFile, NODEDATA_FILE_SUFFIX); // append ".data"
1736             }
1737         }
1738     }
1739     dataFile = tmpDataFile;
1740
1741 }
1742 else
1743 {
1744     // User provided a data file name, but didn't indicate
1745     // if it was a binary data file or not. Make a guess
1746     // based on the file name.
1747
1748     if (strstr(dataFile, HDF_DATA_FILE_SUFFIX) != (char *)NULL)
1749     {
1750         doBinRead = 1;
1751     }
1752 }
1753
1754
1755 PrintBanner(home, argc, argv);
1756
1757 home->ctrlParamList = (ParamList_t *)calloc(1,sizeof(ParamList_t));
1758 home->dataParamList = (ParamList_t *)calloc(1,sizeof(ParamList_t));
1759 home->param          = (Param_t *)calloc(1,sizeof(Param_t ));
1760
1761 param = home->param;
1762
1763 param->doBinRead = doBinRead;
1764 param->maxNumThreads = maxNumThreads;
1765 param->ignoreNonLoopFlux = ignoreNonLoopFlux;
1766

```

```

1767         CtrlParamInit(param, home->ctrlParamList);
1768         DataParamInit(param, home->dataParamList);
1769
1770 #ifdef _BGQ

```

Lines 1583–1585 choose `control.script`. The argument loop (1586–1678) recognizes switches and treats the final non-option argument as the control filename. Lines 1690–1740 derive the data filename, including automatic `.hdf`, `.hdf.0`, and `.data` probing. Lines 1757–1768 allocate both registries and `Param_t`, then call `CtrlParamInit` and `DataParamInit`; this is where defaults are installed before file text is read.

```

1789         // Read runtime parameters from the control file.
1790
1791         printf("Initialize: Parsing control file %s\n", ctrlFile);
1792         ReadControlFile(home, ctrlFile);
1793         printf("Initialize: Control file parsing complete\n");
1794
1795         // Override control file specifications if provided on command line args...
1796
1797         if (param && (maxstep>0))
1798         {
1799             param->maxstep = maxstep;
1800         }
1801
1802         if (param && ((ndx*ndy*ndz)>0))
1803         {
1804             param->nXdoms = ndx;
1805             param->nYdoms = ndy;
1806             param->nZdoms = ndz;
1807         }
1808
1809         if (param && ((ncx*ncy*ncz)>0))
1810         {
1811             param->nXcells = ncx;
1812             param->nYcells = ncy;
1813             param->nZcells = ncz;
1814         }
1815
1816         if (param && dirname)
1817         {
1818             strncpy(param->dirname,dirname,sizeof(param->dirname));
1819         }
1820
1821 #ifdef GPU_ENABLED
1822         if (param && gpu_enabled) { param->gpu_enabled=1; }
1823
1824         if (param)
1825         { printf("GPU-accelerated SegSeg Forces are %s\n", ( param->gpu_enabled ? "ENABLED" : "
1826             DISABLED" ) ); }
1827 #endif
1828
1829         // Set number of initial load-balancing-only cycles. Will
1830         // be zero unless user provided value via "-r" command line
1831         // option.
1832
1833         param->numDLBCycles = numDLBCycles;
1834
1835         // If user explicitly requested major IO to be skipped via
1836         // command line options, override whatever happened to be
1837         // in the control file.

```

```

1837         if (skipIO) param->skipIO = 1;
1838
1839         // Some checks on input consistency
1840
1841         InputSanity(home);

```

Lines 1789–1793 call `ReadControlFile`. Lines 1795–1837 apply CLI values afterward, so `-maxstep`, `-doms`, `-cells`, `-dir`, `-gpu`, `-r`, and `-s` override a control-file assignment. Lines 1839–1841 call `InputSanity`; all cross-field checks happen before MPI propagation.

7 Building the schema: `CtrlParamInit`

```

94 void CtrlParamInit(Param_t *param, ParamList_t *CPList)
95 {
96     // Note: Parameters need only be initialized if their
97     //       default values are non-zero.
98
99     CPList->paramCnt = 0;
100    CPList->varList = (VarData_t *) NULL;
101
102    // Simulation cell and processor setup
103
104    BindVar(CPList, "Simulation cell and processor setup", (void *) NULL, V_COMMENT, 1, VFLAG_NULL);
105
106    BindVar(CPList, "numXdoms", &param->nXdoms, V_INT, 1, VFLAG_NULL); param->nXdoms = 1;
107    BindVar(CPList, "numYdoms", &param->nYdoms, V_INT, 1, VFLAG_NULL); param->nYdoms = 1;
108    BindVar(CPList, "numZdoms", &param->nZdoms, V_INT, 1, VFLAG_NULL); param->nZdoms = 1;
109
110    #if defined _BGQ || defined _BGQ
111        // On BlueGene, user-specified task distribution is used by default;
112        // system does not try to reset it.
113        BindVar(CPList, "taskMappingMode", &param->taskMappingMode, V_INT, 1, VFLAG_NULL); param->
            taskMappingMode = 1;
114    #endif
115
116    BindVar(CPList, "numXcells", &param->nXcells, V_INT, 1, VFLAG_NULL); param->nXcells = 3;
117    // Must be >= 3
118    BindVar(CPList, "numYcells", &param->nYcells, V_INT, 1, VFLAG_NULL); param->nYcells = 3;
119    // Must be >= 3
120    BindVar(CPList, "numZcells", &param->nZcells, V_INT, 1, VFLAG_NULL); param->nZcells = 3;
121    // Must be >= 3
122
123    BindVar(CPList, "xBoundType", &param->xBoundType, V_INT, 1, VFLAG_NULL); param->xBoundType =
        Periodic;
124    BindVar(CPList, "yBoundType", &param->yBoundType, V_INT, 1, VFLAG_NULL); param->yBoundType =
        Periodic;
125    BindVar(CPList, "zBoundType", &param->zBoundType, V_INT, 1, VFLAG_NULL); param->zBoundType =
        Periodic;
126
127    BindVar(CPList, "xBoundMin", &param->xBoundMin, V_DBL, 1, VFLAG_NULL);
128    BindVar(CPList, "xBoundMax", &param->xBoundMax, V_DBL, 1, VFLAG_NULL);
129    BindVar(CPList, "yBoundMin", &param->yBoundMin, V_DBL, 1, VFLAG_NULL);
130    BindVar(CPList, "yBoundMax", &param->yBoundMax, V_DBL, 1, VFLAG_NULL);
131    BindVar(CPList, "zBoundMin", &param->zBoundMin, V_DBL, 1, VFLAG_NULL);
132    BindVar(CPList, "zBoundMax", &param->zBoundMax, V_DBL, 1, VFLAG_NULL);
133
134    BindVar(CPList, "decompType", &param->decompType, V_INT, 1, VFLAG_NULL); param->decompType = 2;
135    BindVar(CPList, "DLBfreq", &param->DLBfreq, V_INT, 1, VFLAG_NULL); param->DLBfreq = 3;

```

```

133
134 // Simulation time and timestepping controls
135
136 BindVar(CPList, "Simulation time and timestepping controls", (void *) NULL, V_COMMENT, 1,
VFLAG_NULL);
137
138 BindVar(CPList, "cycleStart" , &param->cycleStart , V_INT , 1, VFLAG_NULL
);
139 BindVar(CPList, "maxstep" , &param->maxstep , V_INT , 1, VFLAG_NULL
); param->maxstep = 100;
140 BindVar(CPList, "timeNow" , &param->timeNow , V_DBL , 1, VFLAG_NULL
);
141 BindVar(CPList, "timeStart" , &param->timeStart , V_DBL , 1, VFLAG_NULL
);
142 BindVar(CPList, "timeEnd" , &param->timeEnd , V_DBL , 1, VFLAG_NULL
); param->timeEnd = -1.0;
143 BindVar(CPList, "timestepIntegrator" , param->timestepIntegrator , V_STRING, 1, VFLAG_NULL
); strcpy(param->timestepIntegrator, "trapezoid");
144 BindVar(CPList, "trapezoidMaxIterations", &param->trapezoidMaxIterations, V_INT , 1, VFLAG_NULL
); param->trapezoidMaxIterations = 2;
145 BindVar(CPList, "deltaTT" , &param->deltaTT , V_DBL , 1, VFLAG_NULL
);
146 BindVar(CPList, "maxDT" , &param->maxDT , V_DBL , 1, VFLAG_NULL
); param->maxDT = 1.0e-07;
147 BindVar(CPList, "nextDT" , &param->nextDT , V_DBL , 1, VFLAG_NULL
);
148 BindVar(CPList, "dtIncrementFact" , &param->dtIncrementFact , V_DBL , 1, VFLAG_NULL
); param->dtIncrementFact = 1.2;
149 BindVar(CPList, "dtDecrementFact" , &param->dtDecrementFact , V_DBL , 1, VFLAG_NULL
); param->dtDecrementFact = 0.5;
150 BindVar(CPList, "dtExponent" , &param->dtExponent , V_DBL , 1, VFLAG_NULL
); param->dtExponent = 4.0;
151 BindVar(CPList, "dtVariableAdjustment" , &param->dtVariableAdjustment , V_INT , 1, VFLAG_NULL
);
152 BindVar(CPList, "rTol" , &param->rTol , V_DBL , 1, VFLAG_NULL
); param->rTol = -1.0;
153 BindVar(CPList, "rmax" , &param->rmax , V_DBL , 1, VFLAG_NULL
); param->rmax = 100;
154 BindVar(CPList, "subcyclingNumGroups" , &param->subcyclingNumGroups , V_INT , 1, VFLAG_NULL
); param->subcyclingNumGroups = 5;
155 BindVar(CPList, "subcyclingRadii" , param->subcyclingRadii , V_DBL , 4, VFLAG_NULL
);
156 BindVar(CPList, "subcyclingRtolThreshold", &param->subcyclingRtolThreshold, V_DBL, 1, VFLAG_NULL)
; param->subcyclingRtolThreshold = 1.0;
157 BindVar(CPList, "subcyclingRtolRelative", &param->subcyclingRtolRelative, V_DBL , 1, VFLAG_NULL
); param->subcyclingRtolRelative = 0.1;
158 BindVar(CPList, "subcyclingNextDT" , param->subcyclingNextDT , V_DBL , 5, VFLAG_NULL
);
159
160 // GPU controls...
161
162 BindVar(CPList, "gpu_enabled" , &param->gpu_enabled , V_INT , 1, VFLAG_NULL
); param->gpu_enabled = 0; // 0=disable, 1=enable GPU compute kernels
163 BindVar(CPList, "gpu_nstrm" , &param->gpu_nstrm , V_INT , 1, VFLAG_NULL
); param->gpu_nstrm = 4; // default CUDA stream count=4
164 BindVar(CPList, "gpu_nthrd" , &param->gpu_nthrd , V_INT , 1, VFLAG_NULL
); param->gpu_nthrd = 256; // number of threads per kernel launch
165 BindVar(CPList, "gpu_npblk" , &param->gpu_npblk , V_INT , 1, VFLAG_NULL
); param->gpu_npblk = 64*1024; // number of segment pairs per CUDA stream block
166
167 // Sundials/KINSOL controls...

```



```

168
169 BindVar(CPList, "KINSOL_UseNewton" , &param->KINSOL_UseNewton , V_INT , 1,
VFLAG_NULL ); param->KINSOL_UseNewton = 0; // 0=disable, 1=enable newton
170 BindVar(CPList, "KINSOL_MaxIterations" , &param->KINSOL_MaxIterations , V_INT , 1,
VFLAG_NULL ); param->KINSOL_MaxIterations = 5; //
171 BindVar(CPList, "KINSOL_MaxLinIterations" , &param->KINSOL_MaxLinIterations , V_INT , 1,
VFLAG_NULL ); param->KINSOL_MaxLinIterations = 0; //
172 BindVar(CPList, "KINSOL_NumPriorResiduals" , &param->KINSOL_NumPriorResiduals, V_INT , 1,
VFLAG_NULL ); param->KINSOL_NumPriorResiduals = param->KINSOL_MaxIterations-1;
173 BindVar(CPList, "KINSOL_PrintLevel" , &param->KINSOL_PrintLevel , V_INT , 1,
VFLAG_NULL ); param->KINSOL_PrintLevel = 0; // 0=disable
174 BindVar(CPList, "KINSOL_LogFile" , param->KINSOL_LogFile , V_STRING , 1,
VFLAG_NULL ); memset(param->KINSOL_LogFile,0,sizeof(param->KINSOL_LogFile)); // (null string,
must be set within control file)
175 BindVar(CPList, "KINSOL_EtaVal" , &param->KINSOL_EtaVal , V_DBL , 1,
VFLAG_NULL ); param->KINSOL_EtaVal = 0.0; // note: 0.0=forces the value of the
nonlinear residual factor to the KINSOL default to 0.1
176
177 // Sundials/ARKode controls...
178
179 BindVar(CPList, "ARKODE_IRKtable" , &param->ARKODE_IRKtable , V_INT , 1,
VFLAG_NULL ); param->ARKODE_IRKtable = 13; // Billington-3-3-2 Method (3rd Order)
180 BindVar(CPList, "ARKODE_FPsolver" , &param->ARKODE_FPsolver , V_INT , 1,
VFLAG_NULL ); param->ARKODE_FPsolver = 1; // 0=Newton solver, 1=fixed-point solver
181 BindVar(CPList, "ARKODE_MaxNonLinIters" , &param->ARKODE_MaxNonLinIters , V_INT , 1,
VFLAG_NULL ); param->ARKODE_MaxNonLinIters = 4; // 0=use ARKode's default of 3 Newton or
10 FP iterations
182 BindVar(CPList, "ARKODE_MaxLinIters" , &param->ARKODE_MaxLinIters , V_INT , 1,
VFLAG_NULL ); param->ARKODE_MaxLinIters = 0; // 0=use ARKode's default of 5 iterations
183 BindVar(CPList, "ARKODE_FPaccl" , &param->ARKODE_FPaccl , V_INT , 1,
VFLAG_NULL ); param->ARKODE_FPaccl = param->ARKODE_MaxNonLinIters-1; // number of
acceleration vectors
184 BindVar(CPList, "ARKODE_PredictorMethod" , &param->ARKODE_PredictorMethod , V_INT , 1,
VFLAG_NULL ); param->ARKODE_PredictorMethod = 0; // (trivial predictor)
185 BindVar(CPList, "ARKODE_AdaptivityMethod" , &param->ARKODE_AdaptivityMethod , V_INT , 1,
VFLAG_NULL ); param->ARKODE_AdaptivityMethod = 2; // (I step controller)
186 BindVar(CPList, "ARKODE_NonLinCoef" , &param->ARKODE_NonLinCoef , V_DBL , 1,
VFLAG_NULL ); param->ARKODE_NonLinCoef = 1.0; // (nonlinear tolerance factor)
187 BindVar(CPList, "ARKODE_LinCoef" , &param->ARKODE_LinCoef , V_DBL , 1,
VFLAG_NULL ); param->ARKODE_LinCoef = 0.9; // (linear tolerance factor)
188
189 // Discretization controls and controls for topological changes
190
191 BindVar(CPList, "Discretization and topological change controls", (void *) NULL, V_COMMENT, 1,
VFLAG_NULL);
192 BindVar(CPList, "maxSeg" , &param->maxSeg , V_DBL, 1,
VFLAG_NULL); param->maxSeg = -1.0;
193 BindVar(CPList, "minSeg" , &param->minSeg , V_DBL, 1,
VFLAG_NULL); param->minSeg = -1.0;
194 BindVar(CPList, "rann" , &param->rann , V_DBL, 1,
VFLAG_NULL); param->rann = -1.0;
195 BindVar(CPList, "remeshRule" , &param->remeshRule , V_INT, 1,
VFLAG_NULL); param->remeshRule = 2;
196 BindVar(CPList, "splitMultiNodeFreq" , &param->splitMultiNodeFreq , V_INT, 1,
VFLAG_NULL); param->splitMultiNodeFreq = 1;
197 BindVar(CPList, "splitMultiNodeAlpha" , &param->splitMultiNodeAlpha , V_DBL, 1,
VFLAG_NULL); param->splitMultiNodeAlpha = 1.0e-3;
198 BindVar(CPList, "split3node" , &param->split3node , V_INT, 1,
VFLAG_NULL); param->split3node = 1;
199 BindVar(CPList, "useParallelSplitMultiNode", &param->useParallelSplitMultiNode, V_INT, 1,
VFLAG_NULL);

```

```

200 BindVar(CPList, "useUniformJunctionLen" , &param->useUniformJunctionLen , V_INT, 1,
    VFLAG_NULL);
201 BindVar(CPList, "collisionMethod" , &param->collisionMethod , V_INT, 1,
    VFLAG_NULL); param->collisionMethod = 2;

```

Lines 96–100 clear the list. Each BindVar call supplies: the external spelling, destination address, value type, expected count, and flags. The following assignments are defaults. For example, lines 106–108 bind numXdoms to param->nXdoms and default it to one; lines 139–145 register maxstep and timestepIntegrator; lines 191–201 register topology/remesh controls such as remeshRule, splitMultiNodeFreq, and splitMultiNodeAlpha.

```

274 BindVar(CPList, "Loading conditions", (void *) NULL, V_COMMENT, 1, VFLAG_NULL);
275 BindVar(CPList, "TempK" , &param->TempK , V_DBL, 1, VFLAG_NULL); param->TempK =
    600;
276 BindVar(CPList, "pressure" , &param->pressure , V_DBL, 1, VFLAG_NULL);
277 BindVar(CPList, "loadType" , &param->loadType , V_INT, 1, VFLAG_NULL);
278 BindVar(CPList, "appliedStress" , param->appliedStress , V_DBL, 6, VFLAG_NULL);
279 BindVar(CPList, "eRate" , &param->eRate , V_DBL, 1, VFLAG_NULL); param->eRate
    = 1.0;
280 BindVar(CPList, "indxErate" , &param->indxErate , V_INT, 1, VFLAG_NULL); param->
    indxErate = 0;
281 BindVar(CPList, "edotdir" , param->edotdir , V_DBL, 3, VFLAG_NULL); param->edotdir
    [0] = 1.0;
282 param->edotdir
    [1] = 0.0;
283 param->edotdir
    [2] = 0.0;
284
285 BindVar(CPList, "crystalRotation", &param->crystalRotation, V_INT, 1, VFLAG_NULL); param->
    crystalRotation = 1;
286 BindVar(CPList, "cubicModulus" , &param->cubicModulus , V_INT, 1, VFLAG_NULL); param->
    cubicModulus = 0;
287 BindVar(CPList, "cubicC11" , &param->cubicC11 , V_DBL, 1, VFLAG_NULL); param->
    cubicC11 = -1.0;
288 BindVar(CPList, "cubicC12" , &param->cubicC12 , V_DBL, 1, VFLAG_NULL); param->
    cubicC12 = -1.0;
289 BindVar(CPList, "cubicC44" , &param->cubicC44 , V_DBL, 1, VFLAG_NULL); param->
    cubicC44 = -1.0;
290
291 BindVar(CPList, "cTimeOld" , &param->cTimeOld , V_DBL, 1, VFLAG_NULL);
292 BindVar(CPList, "dCyclicStrain" , &param->dCyclicStrain , V_DBL, 1, VFLAG_NULL);
293 BindVar(CPList, "netCyclicStrain", &param->netCyclicStrain, V_DBL, 1, VFLAG_NULL);
294 BindVar(CPList, "numLoadCycle" , &param->numLoadCycle , V_INT, 1, VFLAG_NULL);
295 BindVar(CPList, "eAmp" , &param->eAmp , V_DBL, 1, VFLAG_NULL);
296
297 // To specify input sample axes in laboratory frame
298
299 BindVar(CPList, "useLabFrame" , &param->useLabFrame , V_INT, 1, VFLAG_NULL);
300 BindVar(CPList, "rotationMatrix", param->rotMatrix , V_DBL, 9, VFLAG_NULL);
301
302 // Initialize the rotation matrix to the identity matrix
303
304 for (int row=0; row<3; ++row)
305     for (int col=0; col<3; ++col)
306         param->rotMatrix[row][col] = ( (row==col) ? 1.0 : 0.0 );
307
308 // Parameters for material specific constants and mobility values
309
310 BindVar(CPList, "Material and mobility parameters", (void *) NULL, V_COMMENT, 1, VFLAG_NULL);
311

```

```

312 BindVar(CPList, "mobilityLaw" , param->mobilityLaw , V_STRING, 1, VFLAG_NULL);
    strcpy(param->mobilityLaw , "BCC_0");
313 BindVar(CPList, "materialTypeName" , param->materialTypeName , V_STRING, 1, VFLAG_NULL);
    strcpy(param->materialTypeName, MAT_TYPE_NAME_UNDEFINED);
314 BindVar(CPList, "elasticConstantMatrix", param->elasticConstantMatrix, V_DBL , 36, VFLAG_NULL);

```

Lines 278 and 281 register six-component `appliedStress` and three-component `edotdir`. Lines 300–301 register the nine-component rotation matrix. Lines 312–314 set material/mobility string defaults. These counts are later enforced by `GetParamVals`.

```

560 // I/O controls and options
561
562 BindVar(CPList, "I/O controls and parameters", (void *) NULL, V_COMMENT, 1, VFLAG_NULL);
563 BindVar(CPList, "dirname" , param->dirname , V_STRING, 1, VFLAG_NULL);
564 BindVar(CPList, "writeBinRestart" , &param->writeBinRestart , V_INT, 1, VFLAG_NULL);
565 BindVar(CPList, "skipIO" , &param->skipIO , V_INT, 1, VFLAG_NULL);
566 BindVar(CPList, "numIOGroups" , &param->numIOGroups , V_INT, 1, VFLAG_NULL);
    param->numIOGroups = 1;
567
568 // segment/arm files
569
570 BindVar(CPList, "armfile" , &param->armfile , V_INT, 1, VFLAG_NULL);
571 BindVar(CPList, "armfilefreq" , &param->armfilefreq , V_INT, 1, VFLAG_NULL);
    param->armfilefreq = 100;
572 BindVar(CPList, "armfiledt" , &param->armfiledt , V_DBL, 1, VFLAG_NULL);
    param->armfiledt = -1.0;
573 BindVar(CPList, "armfiletime" , &param->armfiletime , V_DBL, 1, VFLAG_NULL);
574 BindVar(CPList, "armfilecounter" , &param->armfilecounter , V_INT, 1, VFLAG_NULL);
575
576
577 // flux decomposition files
578
579 // Note: the flux related parameters below were renamed, but
580 // we're providing aliases for the original parameter names
581 // so they will still be recognized from older restart files.
582
583 BindVar(CPList, "writeFlux" , &param->writeFlux , V_INT, 1,
    VFLAG_NULL);
584 BindVar(CPList, "writeFluxEdgeDecomp" , &param->writeFluxEdgeDecomp , V_INT, 1,
    VFLAG_NULL); param->writeFluxEdgeDecomp = 1;
585 BindVar(CPList, "writeFluxFullDecomp" , &param->writeFluxFullDecomp , V_INT, 1,
    VFLAG_NULL);
586 BindVar(CPList, "writeFluxFullDecompTotals", &param->writeFluxFullDecompTotals, V_INT, 1,
    VFLAG_NULL);
587 BindVar(CPList, "writeFluxSimpleTotals" , &param->writeFluxSimpleTotals , V_INT, 1,
    VFLAG_NULL);
588 BindVar(CPList, "writeFluxFreq" , &param->writeFluxFreq , V_INT, 1,
    VFLAG_NULL); param->writeFluxFreq = 100;
589 BindVar(CPList, "writeFluxDT" , &param->writeFluxDT , V_DBL, 1,
    VFLAG_NULL); param->writeFluxDT = -1.0;
590 BindVar(CPList, "writeFluxTime" , &param->writeFluxTime , V_DBL, 1,
    VFLAG_NULL);
591 BindVar(CPList, "writeFluxCounter" , &param->writeFluxCounter , V_INT, 1,
    VFLAG_NULL);
592
593 // Aliases
594
595 BindVar(CPList, "fluxfile" , &param->writeFlux , V_INT, 1,
    VFLAG_ALIAS);
596 BindVar(CPList, "fluxfreq" , &param->writeFluxFreq , V_INT, 1,

```

```

VFLAG_ALIAS);
597 BindVar(CPList, "fluxdt" , &param->writeFluxDT , V_DBL, 1,
VFLAG_ALIAS);
598 BindVar(CPList, "fluxtime" , &param->writeFluxTime , V_DBL, 1,
VFLAG_ALIAS);
599 BindVar(CPList, "fluxcounter" , &param->writeFluxCounter , V_INT, 1,
VFLAG_ALIAS);

```

The I/O group binds `dirname`, `binary-restart` and output controls, and defaults `numIOGroups` to one. This is also why changing a field in `Param.h` without adding a `BindVar` does not make a new keyword legal.

```

725 void DataParamInit(Param_t *param, ParamList_t *DPList)
726 {
727     // Note: Parameters need only be initialized if their
728     // default values are non-zero.
729
730     DPList->paramCnt = 0;
731     DPList->varList = (VarData_t *) NULL;
732
733     BindVar(DPList, "dataFileVersion" , &param->dataFileVersion , V_INT, 1, VFLAG_NULL); param->
dataFileVersion = NODEDATA_FILE_VERSION;
734     BindVar(DPList, "numFileSegments" , &param->numFileSegments , V_INT, 1, VFLAG_NULL); param->
numFileSegments = 1;
735     BindVar(DPList, "minCoordinates" , param->minCoordinates , V_DBL, 3, VFLAG_NULL);
736     BindVar(DPList, "maxCoordinates" , param->maxCoordinates , V_DBL, 3, VFLAG_NULL);
737     BindVar(DPList, "nodeCount" , &param->nodeCount , V_INT, 1, VFLAG_NULL);
738     BindVar(DPList, "dataDecompType" , &param->dataDecompType , V_INT, 1, VFLAG_NULL); param->
dataDecompType = 2;
739     BindVar(DPList, "dataDecompGeometry", param->dataDecompGeometry, V_INT, 3, VFLAG_NULL); param->
dataDecompGeometry[0] = 1;
740
param->
dataDecompGeometry[1] = 1;
741
param->
dataDecompGeometry[2] = 1;
742 }

```

`DataParamInit` registers only the restart/data-header keys. Keeping this list separate prevents a nodal-data header from being interpreted as a run-control assignment.

7.1 The binding operation

```

58 void BindVar(ParamList_t *list, const char *name, const void *addr, const int type, const int cnt,
const int flags)
59 {
60     int pCnt;
61     VarData_t *vList;
62
63     vList = list->varList;
64     pCnt = list->paramCnt;
65     list->paramCnt += 1;
66
67     vList = (VarData_t *)realloc(vList, list->paramCnt * sizeof(VarData_t));
68
69     memset(vList[pCnt].varName, 0, MAX_STRING_LEN);
70     strncpy(vList[pCnt].varName, name, MAX_STRING_LEN-1);
71
72     vList[pCnt].valList = (void *) addr;
73     vList[pCnt].valType = type;

```

```

74     vList[pCnt].valCnt = cnt;
75     vList[pCnt].flags  = flags;
76
77     list->varList      = vList;
78
79     return;
80 }

```

Line 63 saves the old array and line 64 records its size. Line 65 increments the count; line 67 reallocates. Lines 69–70 clear and copy the name. Lines 72–75 store the destination pointer, type, count, and flags. Line 77 publishes the enlarged array. Every later parse operation depends on this direct pointer binding.

8 Lexing and parsing, line by line

8.1 ReadControlFile

```

52 void ReadControlFile(Home_t *home, char *ctrlFileName)
53 {
54     int  maxTokenLen, tokenType, pIndex, okay;
55     int  valType, numVals;
56     char token[256];
57     void *vallist;
58     FILE *fpCtrl;
59
60     maxTokenLen = sizeof(token);
61     tokenType = TOKEN_GENERIC;
62
63     if ((fpCtrl = fopen(ctrlFileName, "r")) == (FILE *)NULL) {
64         Fatal("ReadControlFile: Error %d opening file %s",
65             errno, ctrlFileName);
66     }
67
68     while (1) {
69
70         /*
71          *      Next token (if any) should be a parameter name...
72          */
73         tokenType = GetNextToken(fpCtrl, token, maxTokenLen);
74
75         /*
76          *      If we hit the end of the file (i.e. no more tokens)
77          *      we're done...
78          */
79         if (tokenType == TOKEN_NULL) {
80             break;
81         }
82
83         /*
84          *      If there were any parsing errors, just abort
85          */
86         if (tokenType == TOKEN_ERR) {
87             Fatal("ReadControlFile: Error parsing file %s", ctrlFileName);
88         }
89
90         /*
91          *      Obtain values associated with the parameter; don't
92          *      save values for unknown/obsolete parameters.
93          */

```

```

94     pIndex = LookupParam(home->ctrlParamList, token);
95
96     if (pIndex < 0) {
97         printf("Ignoring unrecognized parameter (%s)\n", token);
98         valType = V_NULL;
99         numVals = 0;
100        valList = (void *)NULL;
101    } else {
102        valType = home->ctrlParamList->varList[pIndex].valType;
103        numVals = home->ctrlParamList->varList[pIndex].valCnt;
104        valList = home->ctrlParamList->varList[pIndex].valList;
105        home->ctrlParamList->varList[pIndex].flags |= VFLAG_SET_BY_USER;
106    }
107
108    okay = GetParamVals(fpCtrl, valType, numVals, valList);
109    if (!okay) {
110        Fatal("Error obtaining values for parameter %s",
111            home->ctrlParamList->varList[pIndex].varName);
112    }
113 }
114
115 fclose(fpCtrl);
116
117 return;
118 }

```

Lines 54–61 create the file, token buffer, parser state, and maximum token length. Lines 63–66 open the one requested file and call `Fatal` on failure. The loop begins at line 68. Lines 73–88 fetch a key, stop cleanly at end-of-file, and reject lexical errors. Lines 94–100 perform case-insensitive lookup; an unknown key is reported, then its values are consumed with a null destination so old/new control files remain forward-compatible. Lines 102–105 retrieve the registered type/count/address and mark the entry as user-set. Lines 108–112 convert the value(s), aborting on malformed syntax or wrong cardinality. Line 115 closes the file. Assignments are sequential, so a later occurrence overwrites an earlier one (as demonstrated by the repeated `maxstep` in `tests/bcc_0b.ctrl`).

8.2 Tokenization

```

483 int GetNextToken(FILE *fp, char *token, const int maxTokenSize)
484 {
485     int    inSingleQuote, inDbQuote;
486     int    foundToken, tokenLen, tokenType, appendToToken, nextChar;
487
488     inSingleQuote = 0;
489     inDbQuote     = 0;
490     foundToken    = 0;
491     tokenLen      = 0;
492     tokenType     = TOKEN_GENERIC;
493
494     memset(token, 0, maxTokenSize);
495
496     while (1) {
497
498         /*
499          *      Read the next character from the iunput stream.
500          *      Start with the assumption that the next character is
501          *      part of the token to be returned to the caller.
502          *
503          *      NOTE: In some cases, after reading a character, it

```

```

504  *      is determined the character functions as an end-of-token
505  *      delimiter and we do not want to consume that character
506  *      yet. In these cases, that single character is pushed
507  *      back onto the input stream to be read later.
508  */
509      appendToToken = 1;
510      nextChar = fgetc(fp);
511
512      switch (nextChar) {
513  /*
514  *      White space characters imbedded within quotes are
515  *      considered part of the token. Outside of quotes, they
516  *      are ignored if they precede the next token, and following
517  *      a token they are treated as token delimiters.
518  */
519          case ' ':
520          case '\t':
521          case '\r':
522              if (inSingleQuote || inDbQuote) break;
523              if (foundToken) return(tokenType);
524              appendToToken = 0;
525              break;
526  /*
527  *      Comment characters imbedded within quotes are considered
528  *      part of the token. Outside of quotes they are treated
529  *      as a token delimiter and anything following this character
530  *      up to the next linefeed is ignored.
531  */
532          case '#':
533              if (inSingleQuote || inDbQuote) break;
534              if (foundToken) {
535                  ungetc(nextChar, fp);
536                  return(tokenType);
537              }
538              while ((nextChar != '\0') && (nextChar != '\n')) {
539                  nextChar = fgetc(fp);
540              }
541              if (nextChar == '\n') ungetc(nextChar, fp);
542              appendToToken = 0;
543              break;
544  /*
545  *      Single quotes imbedded within double quotes are considered
546  *      part of the token. Otherwise, quotes are treated as
547  *      token delimiters and any character contained within a
548  *      matched pair of quotes is treated as part of a token.
549  *
550  *      NOTE: Quoted strings may not span multiple lines.
551  */
552          case '\':
553              if (inSingleQuote) return(tokenType);
554              if (inDbQuote) break;
555              if (foundToken) {
556                  ungetc(nextChar, fp);
557                  return(tokenType);
558              }
559              inSingleQuote = 1;
560              foundToken = 1;
561              appendToToken = 0;
562              break;
563  /*
564  *      Double quotes imbedded within single quotes are considered

```

```

565  *      part of the token. Otherwise, quotes are treated as
566  *      token delimiters and any character contained within a
567  *      a matched pair of quotes is treated as part of a token.
568  *
569  *      NOTE: Quoted strings may not span multiple lines.
570  */
571  case '"':
572      if (inDbQuote) return(tokenType);
573      if (inSingleQuote) break;
574      if (foundToken) {
575          ungetc(nextChar, fp);
576          return(tokenType);
577      }
578      inDbQuote = 1;
579      foundToken = 1;
580      appendToToken = 0;
581      break;
582  /*
583  *      '=' characters imbedded within quotes are considered
584  *      part of the token. Outside of quotes, they are
585  *      treated as token delimiters. The function will
586  *      return a zero length string in <token> and a return
587  *      code of TOKEN_EQUAL if this character is the first
588  *      non-white-space character found.
589  */
590  case '=':
591      if (inSingleQuote || inDbQuote) break;
592      if (foundToken) {
593          ungetc(nextChar, fp);
594          return(tokenType);
595      }
596      return(TOKEN_EQUAL);
597  /*
598  *      '[' characters imbedded within quotes are considered
599  *      part of the token. Outside of quotes, they are
600  *      treated as token delimiters. The function will
601  *      return a zero length string in <token> and a return
602  *      code of TOKEN_BEGIN_VAL_LIST if this character is
603  *      the first non-white-space character found.
604  */
605  case '[':
606      if (inSingleQuote || inDbQuote) break;
607      if (foundToken) {
608          ungetc(nextChar, fp);
609          return(tokenType);
610      }
611      return(TOKEN_BEGIN_VAL_LIST);
612  /*
613  *      ']' characters imbedded within quotes are considered
614  *      part of the token. Outside of quotes, they are
615  *      treated as token delimiters. The function will
616  *      return a zero length string in <token> and a return
617  *      code of TOKEN_END_VAL_LIST if this character is
618  *      the first non-white-space character found.
619  */
620  case ']':
621      if (inSingleQuote || inDbQuote) break;
622      if (foundToken) {
623          ungetc(nextChar, fp);
624          return(tokenType);
625      }

```



```

626         return(TOKEN_END_VAL_LIST);
627  /*
628      Line feeds found before the end of a quoted string
629      are considered errors. Outside of a quoted string,
630      they will be treated as token delimiters.
631  */
632         case '\n':
633             if (inSingleQuote || inDbQuote) return(TOKEN_ERR);
634             if (foundToken) return(tokenType);
635             appendToToken = 0;
636             break;
637  /*
638      NULL characters or an EOF found before the end of
639      a quoted string are considered errors. Outside
640      of quoted strings they are treated as token
641      delimiters. If either of these is found before
642      a token, end-of-file is assumed and a return code
643      of TOKEN_NULL is returned to the caller along with
644      a zero length string in <token>.
645  */
646         case 0:
647         case EOF:
648             if (inSingleQuote || inDbQuote) return(TOKEN_ERR);
649             if (foundToken) return(tokenType);
650             return(TOKEN_NULL);
651  /*
652      Anything else is treated as part of the token to be
653      returned to the caller.
654  */
655         default:
656             break;
657     }
658
659  /*
660      If this character is to be treated as part of the token
661      to be returned to the caller, make sure there is enough
662      space available and append the character to the token.
663  */
664     if (appendToToken) {
665         if (++tokenLen > (maxTokenSize-1)) return(TOKEN_ERR);
666         foundToken = 1;
667         tokenType = TOKEN_GENERIC;
668         *token = nextChar;
669         token++;
670     }
671 }
672
673  /*
674      Just for show; we should never hit this.
675  */
676     return(TOKEN_ERR);

```

The tokenizer skips whitespace and # comments (532–543), recognizes quoted strings (552–581), and emits punctuation tokens for =, [, and] (590–626). Quotes may not cross a newline (632–635), end-of-file is explicit (646–650), and the token buffer is bounded at 256 bytes (664–669). This explains why filenames with spaces must be quoted and why comments are allowed only outside a quoted string.

8.3 Lookup and conversion

```

386 int LookupParam(ParamList_t *list, const char *token)
387 {
388     int index1, index2;
389     char *name;
390     void *addr1;
391
392     /*
393     * Compare the specified identifier to the known parameters.
394     * Names are treated as case-insensitive, so if the only
395     * difference between the user-supplied identifier and the
396     * parameter name is in the case of the letters, the strings
397     * are considered equivalent.
398     */
399     for (index1 = 0; index1 < list->paramCnt; index1++) {
400
401         name = list->varList[index1].varName;
402
403         if (StrEquiv(token, name)) {
404             /*
405             * If the specified identifier(token) is actually an
406             * alias for another identifier, find the primary
407             * identifier and return that index instead.
408             *
409             * If the identifier is not an alias for another
410             * identifier OR we cannot determine the identifier
411             * being aliased (which should never happen), return
412             * the index of the current identifier.
413             */
414             if ((list->varList[index1].flags & VFLAG_ALIAS) != 0) {
415                 addr1 = list->varList[index1].valList;
416                 for (index2 = 0; index2 < list->paramCnt; index2++) {
417                     if (index2 == index1) continue;
418                     if (addr1 == list->varList[index2].valList) {
419                         return(index2);
420                     }
421                 }
422             }
423
424             return(index1);
425         }
426     }
427
428     return(-1);
429 }

```

LookupParam compares names with StrEquiv (399–403), so spelling is case-insensitive. Alias records (414–422) resolve to the same destination pointer. The result is an index or -1; the caller decides whether that means “unknown” or a fatal error.

```

239 int GetParamVals(FILE *fp, int valType, int valsExpected, void *valList)
240 {
241     int valCnt, tokenType;
242     int lBracketCnt, isValList, itemCnt;
243     char token[256];
244
245     isValList = 0;
246     lBracketCnt = 0;
247
248     /*

```

```

249  *      Always check for "=" before value or value_list.
250  */
251  tokenType = GetNextToken(fp, token, sizeof(token));
252
253  if (tokenType != TOKEN_EQUAL) {
254      printf("Error: Expected '=' in parameter assignment\n");
255      return(0);
256  }
257
258  /*
259  *      Loop until we've parsed the next value/value_list, or hit
260  *      an error...
261  */
262  for (itemCnt = 0, valCnt = 0; ; valCnt++, itemCnt++) {
263
264      tokenType = GetNextToken(fp, token, sizeof(token));
265
266  /*
267  *      If the first thing we find is a '[', expect a list of
268  *      values contained within []'s.
269  */
270      if ((tokenType == TOKEN_BEGIN_VAL_LIST) && (itemCnt == 0)) {
271          lBracketCnt = 1;
272          isValList = 1;
273          valCnt--;
274          continue;
275      }
276
277  /*
278  *      Take action based on the type of token found
279  */
280      switch (tokenType) {
281
282          case TOKEN_ERR:
283          case TOKEN_NULL:
284              return(0);
285
286  /*
287  *      Nested "[". ignore it but bump up the nesting count
288  */
289          case TOKEN_BEGIN_VAL_LIST:
290              lBracketCnt++;
291              valCnt--;
292              break;
293
294  /*
295  *      Decrement the nesting count for [] pairs. If this
296  *      brings the count negative, there's an error, otherwise
297  *      return to the caller if we found the expected number
298  *      of values or we were parsing an unknown number of values.
299  */
300          case TOKEN_END_VAL_LIST:
301              lBracketCnt--;
302              if (lBracketCnt < 0) return(0);
303              if (lBracketCnt == 0) {
304                  if ((valsExpected == 0) ||
305                      (valCnt == valsExpected)) {
306                      return(1);
307                  }
308                  if (valCnt < valsExpected) return(0);
309              }

```

```

310         valCnt--;
311         break;
312
313     /*
314     *           An "=" is just skipped.
315     */
316     case TOKEN_EQUAL:
317         valCnt--;
318         continue;
319         break;
320
321     /*
322     *           Anything else is treated as a value.
323     */
324     default:
325     /*
326     *           If the caller did not provide a location in which
327     *           to return the results, just continue with the parsing.
328     */
329         if (valList == (void *)NULL) break;
330
331     /*
332     *           If we've already parsed the expected number of values
333     *           there's an error.
334     */
335         if (valCnt >= valsExpected) return(0);
336
337     /*
338     *           Save the value for the caller.
339     */
340     switch (valType) {
341     case V_DBL:
342         ((double *)valList)[valCnt] = atof(token);
343         break;
344     case V_INT:
345         ((int *)valList)[valCnt] = atoi(token);
346         break;
347     case V_STRING:
348         if (valsExpected == 1) {
349             strcpy((char *)valList, token);
350         } else {
351             strcpy(((char **)(valList))[valCnt], token);
352         }
353         break;
354     default:
355         return(0);
356     }
357     break;
358
359     } /* end switch(tokenType) */
360
361     /*
362     *           If we expected and found only a single data item, we're done.
363     */
364     if ((!isValList) && (valsExpected <= 1)) return(1);
365
366     } /* for (valCnt = 0; ... ) */
367 }

```

Line 251 requires =. Lines 262–264 read one or more values. Lines 270–275 enter bracket-list mode. Lines

282–311 reject missing tokens, nested lists, and a closing bracket with the wrong count. Lines 324–357 convert doubles with `atof`, integers with `atoi`, and strings with `strcpy`; line 335 enforces the registered cardinality. A null destination still consumes tokens, which is the mechanism used for unknown names. Because `atoi/atof` are permissive, semantic range and relationship checks belong to `InputSanity`.

9 Precedence and MPI propagation

The effective precedence is:

1. C/C++ zero values; then `CtrlParamInit` defaults.
2. Control-file assignments through `ReadControlFile`.
3. Explicit CLI overrides in `Initialize`.
4. `InputSanity` normalization and derived values.
5. Root-only material/table reads.
6. Raw broadcast of `Param_t`.
7. Nodal-data metadata and post-read derived defaults.

```
1909 #ifdef PARALLEL
1910     MPI_Bcast((char *)param, sizeof(Param_t), MPI_CHAR, 0, MPI_COMM_WORLD);
1911 #endif
```

The three lines broadcast exactly `sizeof(Param_t)` bytes as characters. Registry arrays are not broadcast: they contain process-local pointers. Function pointers are resolved later by `SetMaterialTypeFromMobility`, so raw byte broadcast does not copy invalid function addresses between processes.

```
1971 // Read the nodal data (and associated parameters) from the
1972 // data file (which may consist of multiple file segments).
1973
1974 if (param->doBinRead) { ReadBinDataFile (home, inData, dataFile); }
1975 else { ReadNodeDataFile(home, inData, dataFile); }
1976
1977 // Some of the parameters used in creating the nodal data file
1978 // used for this run may not match the values to be used for
1979 // this run. We've completed processing the data file at this
1980 // point, so update the data file parameters to match the values
1981 // desired for this particular run.
1982
1983 param->dataDecompGeometry[X] = param->nXdoms;
1984 param->dataDecompGeometry[Y] = param->nYdoms;
1985 param->dataDecompGeometry[Z] = param->nZdoms;
1986
1987 param->dataDecompType = param->decompType;
1988 param->dataFileVersion = NODEDATA_FILE_VERSION;
1989 param->numFileSegments = param->numIOGroups;
1990
1991 // Calculate the length of the problem space in each
1992 // of the dimensions
1993
1994 param->Lx = param->maxSideX - param->minSideX;
1995 param->Ly = param->maxSideY - param->minSideY;
1996 param->Lz = param->maxSideZ - param->minSideZ;
1997
1998 // Check which mobility module has been selected and set
1999 // some basic mobility parameters and select the material
2000 // type appropriate to the mobility. This must be done
2001 // before AnisotropicInit() because that function requires
2002 // the proper material type for some initializations.
```

2003
2004

```
SetMaterialTypeFromMobility(home);
```

Lines 1971–1975 select text or binary nodal-data reading. Lines 1983–1989 replace stale metadata with current control values; lines 1994–1996 establish cell lengths. Line 2004 maps the mobility name to executable mobility functions.

10 Validation and derived values

```
22 void InputSanity (Home_t *home)
23 {
24     Param_t *param = home->param;
25
26     int domainCount = param->nXdoms * param->nYdoms * param->nZdoms;
27
28     #ifdef PARALLEL
29     /*
30      *   If we are building for parallel execution, verify that the specified
31      *   geometry matches the domain count...
32      */
33     if (domainCount != home->numDomains)
34     {
35         Fatal("%s; geometry (%dx%dx%d) mismatch with domain count %d\n",
36             __func__, param->nXdoms, param->nYdoms, param->nZdoms, home->numDomains);
37     }
38     #else
39     /*
40      *   If we are building serially, ignore/override the control file domain settings...
41      */
42     if (domainCount != home->numDomains)
43     {
44         Warning("%s() ; ignoring domain geometry (%dx%dx%d) for serial execution",
45             __func__, param->nXdoms, param->nYdoms, param->nZdoms );
46
47         param->nXdoms = 1;
48         param->nYdoms = 1;
49         param->nZdoms = 1;
50     }
51     #endif
52
53     /*
54      *   If the user elects to do parallel IO, insure that the IO
55      *   parallelism does not exceed the number of domains.
56      */
57     if ((param->numIOGroups < 0) || (param->numIOGroups > domainCount)) {
58         printf( "WARNING: The <numIOGroups> control parameter value (%d) "
59             "exceeds the domain count.\n Resetting it to %d\n",
60             param->numIOGroups, domainCount);
61         param->numIOGroups = domainCount;
62     }
63
64     /*
65      *   turn off dynamic load balance if uniprocessor run
66      */
67     if (param->nXdoms * param->nYdoms * param->nZdoms == 1) { param->DLBfreq = 0; }
68
69     /*
70      *   By default we want to dump timing results into a file
```

```

71  *      every time a restart file is created.  If the user did
72  *      not explicitly set variables controlling dumps of timing
73  *      data, go ahead and use the same control values as for
74  *      dumping restarts.
75  */
76      if (param->savetimers < 0)
77          param->savetimers = param->savecn;
78
79      if (param->savetimers > 0) {
80          if (param->savetimersfreq < 1) {
81              if (param->savecnfreq > 0)
82                  param->savetimersfreq = param->savecnfreq;
83              else
84                  param->savetimersfreq = 100;
85          }
86          if (param->savetimerscounter < 1) {
87              if (param->savecnfreq > 0)
88                  param->savetimerscounter = param->savecncounter;
89              else
90                  param->savetimerscounter = 1;
91          }
92          if (param->savetimersdt < 0.0) {
93              if (param->savecndt > 0.0) {
94                  param->savetimersdt = param->savecndt;
95                  param->savetimerstime = param->savecntime;
96              } else {
97                  param->savetimersdt = 0.0;
98                  param->savetimerstime = 0.0;
99              }
100          }
101      }
102
103  /*
104  *      If there are no Eshelby inclusions being simulated, explicitly turn
105  *      off the eshelby FMM (whether it was on or not).
106  */
107  #ifdef ESHELBY
108      if (param->inclusionFile[0] == 0) {
109          param->eshelbyfmEnabled = 0;
110      }
111  #endif
112
113  /*
114  *      The current implementation of the fast-multipole code to deal
115  *      with remote seg forces requires that the number of cells
116  *      be identical in each dimension as well as being a power of 2.
117  *      Here we just enforce that.
118  */
119  #ifdef ESHELBY
120      if ((param->fmEnabled) || (param->eshelbyfmEnabled))
121  #else
122      if (param->fmEnabled)
123  #endif
124      {
125          if ((param->nXcells != param->nYcells) ||
126              (param->nXcells != param->nZcells))
127          {
128              Fatal("Cell geometry (%dX%dX%d) invalid.  %s",
129                  param->nXcells, param->nYcells, param->nZcells,
130                  "Number of cells must be identical in each dimension");
131          }

```

```

132         for (int i=2; (i<10); i++)
133         {
134             int cellCount = 1 << i;
135
136             if (param->nXcells == cellCount)
137             {
138                 param->fmNumLayers = i+1;
139                 break;
140             }
141
142             if (param->nXcells < cellCount)
143                 Fatal("Number of cells in a dimension must be at least 4 and a power of 2");
144         }
145     }
146 }
147
148 /*
149  *   If the fast multipole code is enabled for remote force
150  *   calculations, the number of points along each segment
151  *   at which to evaluate stress from the expansions
152  *   is a function of the expansion order... but don't
153  *   go higher than 6.
154  */
155 #ifdef ESHELBY
156     if ((param->fmEnabled) || (param->eshelbyfmEnabled))
157 #else
158     if (param->fmEnabled)
159 #endif
160     {
161         param->fmNumPoints = (param->fmExpansionOrder / 2) + 1;
162         param->fmNumPoints = MIN(param->fmNumPoints, 6);
163     }
164
165 /*
166  *   Verify the domain decomposition type specified is valid.
167  */
168     if ((param->decompType < 1) || (param->decompType > 2)) {
169         Fatal("decompType=%d is invalid. Type must be 1 or 2\n", param->decompType);
170     }

```

The early lines validate domain counts, MPI geometry, I/O-group counts, FMM cell/layer settings, and decomposition. Importantly, checks combine fields: a syntactically valid value can still be rejected or clamped when it conflicts with another parameter.

```

227     if ( (param->loadType==0) && (param->eRate!=1.0) )
228     {
229         printf("Warning: eRate=%le is inconsistent with loadType=0, setting eRate to 1.0\n",
230             param->eRate);
231         param->eRate = 1.0;
232     }
233
234 /*
235  *   Make sure the frequency at which to do multi-node splits is > 0.
236  */
237     param->splitMultiNodeFreq = MAX(1, param->splitMultiNodeFreq);
238     param->split3node = (param->split3node != 0);

```

Load type is checked and split frequency is clamped to a legal positive value; boolean topology flags are normalized. The same function contains specialized checks for trapezoid integration, subcycling, KINSOL, anisotropic elasticity, and HCP constraints.


```

392 void SetMaterialTypeFromMobility(Home_t *home)
393 {
394     Param_t *param    = home->param;
395     int      mobIndex = 0;
396     int      segmentResistAlias =
397         StrEquiv(param->mobilityLaw,
398             "BCC_nl_Eshelby_SegmentResist");
399
400     // Look up the attributes of the user-specified mobility module
401     // and set various mobility parameters appropriately.
402
403     for (mobIndex=0; mobIndex < MOB_MAX_INDEX; mobIndex++)
404     {
405         int mat=0;
406
407         if (StrEquiv(param->mobilityLaw,
408             (char *)mobAttrList[mobIndex].mobName) ||
409             (segmentResistAlias &&
410             StrEquiv((char *)mobAttrList[mobIndex].mobName,
411                 "BCC_nl_Eshelby_Resist")))
412         {
413             param->mobilityType      = mobAttrList[mobIndex].mobilityType;
414             param->mobilityInitFunc   = mobAttrList[mobIndex].mobInitFunc;
415             param->mobilityFunc       = mobAttrList[mobIndex].mobFunc;
416
417             // If the mobility module is planar and glide planes are
418             // not being enforced, explicitly enable the enforcement
419             // and warn the user.
420
421             if ((mobAttrList[mobIndex].isPlanarMobility) &&
422                 (param->enforceGlidePlanes == 0)) {
423
424                 param->enforceGlidePlanes = 1;
425
426                 if (home->myDomain == 0) {
427                     printf("The specified mobility (%s) requires the "
428                         "enforceGlidePlanes\ncontrol parameter "
429                         "toggle to be set. Enabling toggle now.\n",
430                         param->mobilityLaw);
431                 }
432             }
433
434             if ((mobAttrList[mobIndex].isPlanarMobility == 0) &&
435                 (param->enforceGlidePlanes == 1)) {
436
437                 param->enforceGlidePlanes = 0;
438
439                 if (home->myDomain == 0) {
440                     printf("The specified mobility (%s) does not allow the "
441                         "enforceGlidePlanes feature.\n So the control parameter "
442                         "should be set to be zero.\n",
443                         param->mobilityLaw);
444                 }
445             }
446
447             param->allowFuzzyGlidePlanes = mobAttrList[mobIndex].allowFuzzyPlanes;
448
449             // Map the user-specified material type name (if any) to
450             // the corresponding integer value. If the user did not
451             // specify a material type, use the default type for the

```

```

452         // mobility module selected.
453
454         if (StrEquiv(param->materialTypeName, MAT_TYPE_NAME_UNDEFINED)){
455             int matType = mobAttrList[mobIndex].defaultMatType;
456             for (mat = 0; mat < MAT_TYPE_MAX_INDEX; mat++) {
457                 if (matType == matInfo[mat].materialType) {
458                     param->materialType = matType;
459                     param->numBurgGroups = matInfo[mat].numBurgGroups;
460                     strcpy(param->materialTypeName,
461                           matInfo[mat].materialTypeName);
462                     break;
463                 }
464             }
465         } else {
466             for (mat = 0; mat < MAT_TYPE_MAX_INDEX; mat++) {
467                 if (StrEquiv(param->materialTypeName,
468                             (char *)matInfo[mat].materialTypeName)) {
469
470                     param->materialType = matInfo[mat].materialType;
471                     param->numBurgGroups = matInfo[mat].numBurgGroups;
472                     break;
473                 }
474             }
475         }
476
477         if (mat >= MAT_TYPE_MAX_INDEX) {
478             Fatal("Specified <materialTypeName> (%s) is invalid.", param->materialTypeName);
479         }
480
481         // If the user-specified material type does not match the
482         // default material type for the mobility module, we *may*
483         // want to warn the user.
484
485         if (param->materialType != mobAttrList[mobIndex].defaultMatType) {
486             if (mobAttrList[mobIndex].warnOnMatTypeMismatch) {
487                 printf("Warning: <materialType=%s> does not match the"
488                       "\n          default material type for "
489                       "<mobilityLaw=%s>\n", matInfo[mat].materialTypeName, param->
mobilityLaw);
490             }
491         }
492
493         param->mobilityIndex = mobIndex;
494         break;
495     }
496 }
497
498 if (mobIndex >= MOB_MAX_INDEX) {
499     Fatal("Unknown mobility function %s", param->mobilityLaw);
500 }
501
502 return;
503 }

```

Lines 392–399 obtain the parameter object. Lines 403–415 scan the mobility table and copy the selected law’s enum and function pointers. Lines 421–445 enforce glide-plane capabilities; lines 454–475 infer material type and Burgers-group count; lines 477–500 reject incompatible or unknown mobility names.

```

544 void SetRemainingDefaults(Home_t *home)
545 {

```

```

546     real8    tmp, eps;
547     real8    xCellSize, yCellSize, zCellSize, minCellSize;
548     Param_t  *param;
549
550     param = home->param;
551
552     param->delSegLength = 0.0;
553
554     xCellSize = param->Lx / param->nXcells;
555     yCellSize = param->Ly / param->nYcells;
556     zCellSize = param->Lz / param->nZcells;
557
558     minCellSize = MIN(xCellSize, yCellSize);
559     minCellSize = MIN(minCellSize, zCellSize);
560
561     eps = 1.0e-02;
562
563     // The core radius and maximum segment length are required
564     // inputs. If the user did not provide both values, abort
565     // now.
566
567     if (home->myDomain == 0) {
568         if (param->rc < 0.0) {
569             Fatal("The <rc> parameter is required but was not \n"
570                 "    provided in the control file");
571         }
572
573         if (param->maxSeg < 0.0) {
574             Fatal("The <maxSeg> parameter is required but was not \n"
575                 "    provided in the control file");
576         }
577     }
578
579     // If not provided, set position error tolerance based on <rc>
580
581     if (param->rTol <= 0.0) {
582         param->rTol = 0.25 * param->rc;
583     }
584
585     // The deltaTT is set in the timestep integrator, but some
586     // mobility functions now use the deltaTT value, so it must
587     // be initialized before ParadisStep() is called since there
588     // is an initial mobility calculation done *before* timestep
589     // integration the first time into the function.
590
591     param->deltaTT = MIN(param->maxDT, param->nextDT);
592
593     if (param->deltaTT <= 0.0) {
594         param->deltaTT = param->maxDT;
595     }
596
597     // If the annihilation distance has not been provided, set
598     // it to a default based on <rc>
599
600     if (param->rann <= 0.0) {
601         param->rann = 2.0 * param->rTol;
602     }
603
604     // Minimum area criteria for remesh is dependent on maximum
605     // and minnumum segment lengths and position error tolerance.
606

```

```

607 param->remeshAreaMin = 2.0 * param->rTol * param->maxSeg;
608
609 if (param->minSeg > 0.0) {
610     param->remeshAreaMin = MIN(param->remeshAreaMin,
611                               (param->minSeg * param->minSeg *
612                                sqrt(3.0) / 4));
613 }
614
615 // Maximum area criteria for remesh is dependent on minimum area,
616 // and maximum segment length.
617
618 param->remeshAreaMax = 0.5 * ((4.0 * param->remeshAreaMin) +
619                               (0.25 * sqrt(3)) *
620                               (param->maxSeg*param->maxSeg));
621
622 // If the user did not provide a minSeg length, calculate one
623 // based on the remesh minimum area criteria.
624
625 if (param->minSeg <= 0.0) {
626     param->minSeg = sqrt(param->remeshAreaMin * (4.0 / sqrt(3)));
627 }
628
629 /*
630  * Generate ExaDiS-style default separating radii when the user did
631  * not provide any. The first value remains zero in the control
632  * state; the drift driver additionally places hinges (and distances
633  * below one Burgers vector) in group zero.
634  */
635 if (StrEquiv(param->timestepIntegrator, "subcycling")) {
636     int haveRadii = 0;
637     int numRadii = param->subcyclingNumGroups - 1;
638
639     for (int i = 0; i < numRadii; i++) {
640         haveRadii |= (param->subcyclingRadii[i] != 0.0);
641     }
642
643     if (!haveRadii) {
644         real8 rMax = param->maxSeg;
645         real8 rMin = MIN(MAX(0.3 * param->minSeg,
646                             3.0 * param->rann), rMax);
647
648         param->subcyclingRadii[0] = 0.0;
649
650         for (int i = 1; i < numRadii; i++) {
651             real8 phase = (((real8)(i - 1) /
652                           (param->subcyclingNumGroups - 2)) -
653                           0.5) * M_PI;
654             real8 fraction = 0.5 * (sin(phase) + 1.0);
655             param->subcyclingRadii[i] =
656                 rMin + fraction * (rMax - rMin);
657         }
658     }
659
660     for (int i = 0; i < 5; i++) {
661         if (param->subcyclingNextDT[i] <= 0.0) {
662             param->subcyclingNextDT[i] = param->deltaTT;
663         }
664     }
665 }
666
667 // If the user did not provide an Ecore value, set the default

```

```

668 // based on the shear modulus and rc values
669
670 if (param->Ecore < 0.0) {
671     param->Ecore = (param->shearModulus / (4*M_PI)) *
672         log(param->rc/0.1);
673 }
674
675 #ifndef ESHELBY
676     if (param->shearModulus2 < 0.0) param->shearModulus2 = param->shearModulus;
677     if (param->pois2 < 0.0) param->pois2 = param->pois;
678     if (param->Ecore2 < 0.0) param->Ecore2 = (param->shearModulus2/(4*M_PI)) * log(
        param->rc/0.1);
679 #endif
680 // Now do some additional sanity checks.
681
682 if (home->myDomain == 0) {
683
684     // First check for some fatal errors...
685
686     if (param->maxSeg <= param->rTol * (32.0 / sqrt(3.0))) {
687         Fatal("Maximum segment length must be > rTol * 32 / sqrt(3)\n"
688             "    Current maxSeg = %lf, rTol = %lf",
689             param->maxSeg, param->rTol);
690     }
691
692     if (param->minSeg > (0.5 * param->maxSeg)) {
693         Fatal("Minimum segment length must be < (0.5 * maxSeg)\n"
694             "    Current minSeg = %lf, maxSeg = %lf",
695             param->minSeg, param->maxSeg);
696     }
697
698     if (param->maxSeg <= param->minSeg) {
699         Fatal("Max segment length (%e) must be greater than the\n"
700             "    minimum segment length (%e)", param->maxSeg,
701             param->minSeg);
702     }
703
704     if (param->maxSeg > (minCellSize * 0.9)) {
705         Fatal("The maxSeg length must be less than the "
706             "minimum cell size * 0.9. Current values:\n"
707             "    maxSeg = %lf\n    cellWidth = %lf",
708             param->maxSeg, minCellSize);
709     }
710
711     if (param->remeshAreaMin > (0.25 * param->remeshAreaMax)) {
712         Fatal("remeshAreaMin must be less than 0.25*remeshAreaMax\n"
713             "    Current remeshAreaMin = %lf, remeshAreaMax = %lf",
714             param->remeshAreaMin, param->remeshAreaMax);
715     }

```

`SetRemainingDefaults` runs after nodal data. It derives cell lengths, cutoff and segment limits, tolerances, subcycling values, core energy, periodic bounds, density storage, and volume. Sentinel values such as a negative `maxSeg` are therefore intentionally resolved here, not in the text parser.

11 The nodal data/restart boundary

```

466 void ReadNodeDataFile(Home_t *home, InData_t *inData, char *dataFile)
467 {

```

```

468     int        i, dom, iNbr;
469     int        maxTokenLen, tokenType, pIndex, okay;
470     int        valType, numVals;
471     int        numDomains, numReadTasks;
472     int        nextFileSeg, maxFileSeg, segsPerReader, taskIsReader;
473     int        distIncomplete, readCount, numNbrs;
474     int        fileSegCount, fileSeqNum = 0;
475     int        *globalMsgCnt, *localMsgCnt, **nodeLists, *listCounts;
476     int        miscData[2]={0,0};
477     int        localNodeCount, globalNodeCount;
478     int        binRead = 0;
479     int        nextAvailableTag = 0;
480     real8      burgSumX, burgSumY, burgSumZ;
481     void        *vallList;
482     char        inLine[500], token[256];
483     char        baseFileName[256], tmpFileName[256];
484     FILE        *fpSeg;
485     Node_t      *node;
486     Param_t     *param;
487     ParamList_t *dataParamList;
488
489     param        = home->param;
490     numDomains   = home->numDomains;
491     fpSeg        = (FILE *)NULL;
492
493     dataParamList = home->dataParamList;
494
495     globalMsgCnt = home->globalReduceBuf;
496     localMsgCnt  = home->localReduceBuf;
497
498     memset(inLine, 0, sizeof(inLine));
499     maxTokenLen = sizeof(token);
500
501     /*
502      *      Only domain zero reads the initial stuff...
503      */
504     if (home->myDomain == 0) {
505
506         if (dataFile == (char *)NULL) {
507             Fatal("ReadNodeDataFile: No data file provided");
508         }
509
510         snprintf(tmpFileName, sizeof(tmpFileName), "%s", dataFile);
511
512         if (!strcmp(&tmpFileName[strlen(tmpFileName)-2], ".0")) {
513             tmpFileName[strlen(tmpFileName)-2] = 0;
514         }
515
516         snprintf(baseFileName, sizeof(baseFileName), "%s", tmpFileName);
517
518         /*
519          *      Try to open the nodal data file. If the specified file
520          *      can not be opened, try looking for a segmented data file
521          *      (i.e. look for the first file segment <dataFile>.0). If
522          *      that can't be opened either, then exit with an error.
523          */
524
525         if ((fpSeg = fopen(tmpFileName, "r")) == (FILE *)NULL) {
526             snprintf(tmpFileName, sizeof(tmpFileName), "%s.%d",
527                     baseFileName, fileSeqNum);
528             if ((fpSeg = fopen(tmpFileName, "r")) == (FILE *)NULL) {

```

```

529         Fatal("Error %d opening file %s to read nodal data",
530               errno, dataFile);
531     }
532 }
533
534 /*
535  *      Get the first token. This should either be a known
536  *      parameter identifier, or a file version number.
537  */
538 tokenType = GetNextToken(fpSeg, token, maxTokenLen);
539 pIndex = LookupParam(dataParamList, token);
540
541 if (pIndex < 0) {
542     /*
543      *      If the token does not correspond to a known parameter, it
544      *      should be the version number, so convert it to an integer
545      *      and parse the rest of the file the old way (i.e. values
546      *      are positional, not identified by name/value pairs)
547      */
548     param->dataFileVersion = atoi(token);
549
550     if ((param->dataFileVersion < 1) ||
551         (param->dataFileVersion > 3)) {
552         Fatal("ReadNodeDatFile: Unsupported file version %d",
553               param->dataFileVersion);
554     }
555
556     ReadPreV4DataParams(home, fpSeg, &inData->decomp);
557
558 } else {
559     /*
560      *      Just go through the nodal data file reading all
561      *      the associated parameters. Need to do special
562      *      processing of the domain decomposition, and when
563      *      when we hit the 'nodaldata' identifier, just break
564      *      out of this loop.
565      */
566     while ((tokenType != TOKEN_ERR) && (tokenType != TOKEN_NULL)) {
567
568         if (pIndex >= 0) {
569             /*
570              *      Token represents a known parameter identifier, so
571              *      read the associated value(s).
572              */
573             valType = dataParamList->varList[pIndex].valType;
574             numVals = dataParamList->varList[pIndex].valCnt;
575             vallist = dataParamList->varList[pIndex].vallist;
576             okay = GetParamVals(fpSeg, valType, numVals, vallist);
577             if (!okay) {
578                 Fatal("Parsing Error obtaining values for "
579                       "parameter %s\n",
580                       dataParamList->varList[pIndex].varName);
581             }
582
583         } else {
584             /*
585              *      Token does not represent one of the simple
586              *      parameters. If it's not one of the identifiers
587              *      that needs special handling, skip it.
588              */
589             if (strcmp(token, "domainDecomposition") == 0) {

```

```

590  /*
591  *      The minSide{XYZ} and maxSide{XYZ} values are
592  *      now redundant but until the rest of the code
593  *      is modified to remove references to these
594  *      values, we need to explicitly set them now.
595  */
596      param->minSideX = param->minCoordinates[X];
597      param->minSideY = param->minCoordinates[Y];
598      param->minSideZ = param->minCoordinates[Z];
599
600      param->maxSideX = param->maxCoordinates[X];
601      param->maxSideY = param->maxCoordinates[Y];
602      param->maxSideZ = param->maxCoordinates[Z];
603
604      tokenType = GetNextToken(fpSeg, token, maxTokenLen);
605  /*
606  *      Do a quick verification of the decomposition type
607  */
608      if ((param->dataDecompType < 1) ||
609          (param->dataDecompType > 2)) {
610          Fatal("dataDecompType=%d is invalid.  Type must be 1 or 2\n",
611              param->dataDecompType);
612      }
613
614      ReadDecompBounds(home, (void **)&fpSeg, binRead,
615                      param->decompType,
616                      &inData->decomp);
617
618      } else if (strcmp(token, "nodalData") == 0) {
619  /*
620  *      When we hit the nodal data, we can break
621  *      out of the loop because we are assuming
622  *      all other data file parameters have been
623  *      processed.  If they have not, we have a
624  *      problem since processing of the nodal data
625  *      requires the other parameters.
626  *      Note: Remainder of the file should just
627  *      contain " = " followed by the nodal data,
628  *      so be sure to skip the next token before
629  *      processing the nodal data.
630  */
631      tokenType = GetNextToken(fpSeg, token, maxTokenLen);
632      break;
633  } else {
634  /*
635  *      If the parameter is not recognized, skip the
636  *      parameter and any associated value(s).
637  */
638      printf("Ignoring unknown data file parameter %s\n",
639          token);
640      valType = V_NULL;
641      numVals = 0;
642      vallist = (void *)NULL;
643      okay = GetParamVals(fpSeg, valType, numVals,
644                          vallist);
645  }
646  }
647
648      tokenType = GetNextToken(fpSeg, token, maxTokenLen);
649
650      if ((tokenType == TOKEN_NULL) || (tokenType == TOKEN_ERR)) {

```



```

651         Fatal("Parsing error on file %s\n", tmpFileName);
652     }
653     pIndex = LookupParam(dataParamList, token);
654 }
655 }

```

`ReadNodeDataFile` opens the data file, reads its header, and loops over the separate data registry. `domainDecomposition` is special: its min/max coordinates can be converted into legacy decomposition bounds. The `nodalData` marker ends the header; node records follow. Binary restart reading performs the analogous job in `src/ReadBinaryRestart.cc`. This path supplies node state and file geometry; it does not replace the control-file registry.

12 From Param_t to the first simulation cycle

12.1 Cycle count and integrator

```

320     firstTime      = 0;
321 }
322 else
323 {
324     nodeForceLevel = PARTIAL;
325     zeroOnError    = 0;
326 #ifdef ESHELBY
327     doAllVelocities = 0;
328 #else
329     doAllVelocities = 1;
330 #endif
331 }
332
333 NodeForce      (home, nodeForceLevel);
334 CalcNodeVelocities(home, zeroOnError, doAllVelocities);
335 CommSendVelocity (home);
336
337 // Invoke the selected time step integration method. The
338 // selected method will calculate the time step as well as
339 // move nodes to their correct locations and communicate
340 // the new nodal force/velocity data to neighboring domains.
341 //
342 // The 'trapezoid' method used to be specified as 'backard-euler', so
343 // if the integration method is unrecognized, just use the trapezoid
344 // method as default
345
346     if (StrEquiv(param->timestepIntegrator, "forward-euler" )) { ForwardEulerIntegrator (
347         home); }
347     else if (StrEquiv(param->timestepIntegrator, "trapezoid" )) { TrapezoidIntegrator      (
348         home); }
348     else if (StrEquiv(param->timestepIntegrator, "trapezoid-multi" )) { TrapezoidIntegratorMulti (
349         home); }
349     else if (StrEquiv(param->timestepIntegrator, "trapezoid-kinsol" )) { TrapezoidIntegratorKINSOL(
350         home); }
350     else if (StrEquiv(param->timestepIntegrator, "arkode" )) { ARKodeIntegrator          (
351         home); }
351     else if (StrEquiv(param->timestepIntegrator, "subcycling" )) { SubcyclingIntegrator    (
352         home); }
352     else { TrapezoidIntegrator      (
353         home); } ///< (default)

```

The step routine computes forces/velocities and dispatches on `timestepIntegrator`. A string such as `trapezoid` therefore changes the integrator branch, while the main loop's `maxstep` controls how many times this routine is entered.

```

784 void TrapezoidIntegrator(Home_t *home)
785 {
786
787 #ifdef DEBUG_TIMESTEP_ERROR
788     int      dumpErrorData=1;
789 #endif
790
791     Node_t *maxNode = 0;           // maxNode = points to node with largest position error
792     Node_t *posNode = 0;          // posNode = points to node with largest change in
    position
793
794     Param_t *param = home->param;
795
796     real8 oldDT = param->deltaTT;
797     real8 newDT = MIN(param->maxDT, param->nextDT);
798
799     if (newDT <= 0.0) newDT = param->maxDT;
800
801     param->deltaTT = newDT;
802
803     // Preserve certain items of nodal data for later use.
804
805     Preserve_Nodes (home->nodeKeys      , home->newNodeKeyPtr , NODE_POSITION | NODE_CURR_VEL );
806     Preserve_Nodes (home->ghostNodeList, home->ghostNodeCount, NODE_POSITION | NODE_CURR_VEL );
807
808     // Loop until we converge on a time step. First step is to
809     // use the current positions and velocities and reposition the
810     // nodes to where they would be after the suggested delta time.
811
812     int  convergent      = 0;
813     int  incrDelta       = 1;
814     int  maxIterations   = param->trapezoidMaxIterations;
815
816     real8 errMax         = 0.0;
817     real8 posMax         = 0.0;
818     real8 globalErrMax   = 0.0;
819     real8 globalPosMax   = 0.0;
820     int  globalIterError = 0;
821
822     while (!convergent)
823     {

```

The trapezoid integrator copies `deltaTT` into a trial step, bounds it by `maxDT`, and uses `trapezoidMaxIterations`. Later code (904–1000) accepts/rejects the trial using `rTol` and `maxSeg`, then adjusts the next step with `dtDecrementFact`, `dtIncrementFact`, `dtExponent`, and `dtVariableAdjustment`.

12.2 Topology, remeshing, and cross slip

```

379 // Before doing topological changes, set flags indicating any
380 // nodes exempt from topological changes. These flags are used
381 // in both splitting multi-arm nodes and collisions, so this
382 // function should be invoked before either of those items are done.
383
384 InitTopologyExemptions(home);
385

```

```

386 // Now do all the topological changes from segment interactions
387 // (collisions, multinode splitting)... Clear the list of local
388 // operations that will be sent to the remote domains for processing,
389 // then split any multi-arm nodes that need splitting, cross slip
390 // nodes (as needed/allowed), handle all local collisions, then
391 // send remote nodes the list of ops needed to keep their data in sync.
392
393 ClearOpList(home);
394 SortNodesForCollision(home);
395
396 #ifdef FIX_PLASTIC_STRAIN
397 // Set the plastic strain correction to zero before handling
398 // the topological operations.
399
400 ResetPlasticStrain(home);
401 #endif
402
403 if (param->useParallelSplitMultiNode) { SplitMultiNodesParallel(home); }
404 else { SplitMultiNodes (home); }
405
406 #ifdef _ARLFEM
407 SplitSurfaceNodes(home);
408 #endif
409
410 // Call a generic cross-slip dispatch function that will invoke
411 // (if necessary) the cross-slip function appropriate to the
412 // type of material in use.
413
414 CrossSlip(home);
415
416 // Search for dislocation segments in close proximity to each other
417 // and if necessary handle any collision between them.
418
419 HandleCollisions(home);
420
421 // Check for any segments that should be decomposed (splintered)
422 // into multiple segments of a different burgers vector.
423
424 SplinterSegments(home);
425
426 // For HCP simulations, we may want to split segments with <c+a>
427 // burgers into two segments with a <c> and an <a> burgers vector
428 // respectively, and cross-slip one to a new plane...
429
430 HCP_CA_Split(home);
431
432 #ifdef ESHELBY
433 // Handle collisions between dislocations and particles
434
435 if (home->param->RefineSegCollision == 1)
436     HandleParticleCollisions(home);
437 #endif
438
439 #ifdef PARALLEL
440 #ifdef SYNC_TIMERS
441 TimerStart (home, POST_COLLISION_BARRIER);
442 MPI_Barrier(MPI_COMM_WORLD);
443 TimerStop (home, POST_COLLISION_BARRIER);
444 #endif
445 #endif
446 TimerStart (home, COL_SEND_REMESH);

```

```

447     CommSendRemesh(home);
448     TimerStop      (home, COL_SEND_REMESH);
449
450     TimerStart      (home, COL_FIX_REMESH );
451     FixRemesh       (home);
452     TimerStop      (home, COL_FIX_REMESH );
453
454     // For debugging purposes only, the following check has been added
455     // to force an immediate abort if we detect an unconstrained node
456     // at which the burgers vector is not conserved. This code can
457     // be removed once all the above topology modifying functions
458     // have been verified to play well together.
459
460     CheckBurgConservation(home, "After FixRemesh()");
461
462     #ifdef _ARLFEM
463         FixSurfaceTopology(home);
464     #endif
465
466     // Under certain circumstances, parallel topological changes can
467     // create double links between nodes; links which can not be detected
468     // until after FixRemesh() is called... so, a quick check has to be
469     // done to clean up these potential double-links here, or they will
470     // cause problems later on. Should only have to check nodes local
471     // to this domain.
472
473     for (int i=0; i < home->newNodeKeyPtr; i++)
474     {
475         Node_t *node = home->nodeKeys[i];
476
477         if (node)
478         {
479             RemoveDoubleLinks(home, node, 0);
480             node->flags &= ~NODE_CHK_DBL_LINK;
481         }
482     }
483
484     // If memory debugging is enabled, run a consistency check on all
485     // allocated memory blocks looking for corruption in any of the
486     // block headers or trailers.
487
488     #ifdef DEBUG_MEM
489         ParadisMemCheck();
490     #endif
491     // Invoke mesh coarsen/refine
492
493     Remesh(home);

```

This region gates topology work, parallel remeshing, cross slip, and the final remesh call.

```

1385  /*
1386  *      Only do multinode splits on cycles that are multiples
1387  *      of the <splitMultiNodeFreq> control file parameter.
1388  */
1389      if (home->cycle % home->param->splitMultiNodeFreq) {
1390          skipSplits = 1;
1391      }
1392
1393      TimerStart(home, SPLIT_MULTI_NODES);
1394
1395      param      = home->param;

```

```

1396     minDist    = param->rann * 2.0 + eps;
1397     shortSeg   = MIN(5.0, param->minSeg * 0.1);
1398     vNoise     = param->splitMultiNodeAlpha * param->rTol / param->deltaTT;
1399     splitDist  = minDist;

```

`splitMultiNodeFreq` determines how often multi-node splitting is attempted; `rann`, `minSeg`, `splitMultiNodeAlpha`, `rTol`, and `deltaTT` determine the geometric and tolerance tests.

```

1172 void Remesh(Home_t *home)
1173 {
1174     int i;
1175     Node_t *node;
1176     Param_t *param;
1177
1178     param = home->param;
1179
1180     if (param->remeshRule < 0) return;
1181
1182     #ifdef PARALLEL
1183     #ifdef SYNC_TIMERS
1184         TimerStart(home, REMESH_START_BARRIER);
1185         MPI_Barrier(MPI_COMM_WORLD);
1186         TimerStop(home, REMESH_START_BARRIER);
1187     #endif
1188     #endif
1189
1190     TimerStart(home, REMESH);
1191     ClearOpList(home);
1192     InitTopologyExemptions(home);
1193
1194     switch(param->remeshRule) {
1195     case 2:
1196         RemeshRule_2(home);
1197         break;
1198     case 3:
1199         RemeshRule_3(home);
1200         break;
1201     default:
1202         Fatal("Remesh: undefined remesh rule %d", param->remeshRule);
1203         break;
1204     }

```

Negative `remeshRule` disables remeshing. Rules 2 and 3 select the corresponding remesh algorithms.

```

636 void CrossSlip(Home_t *home)
637 {
638     int enabled = home->param->enableCrossSlip;
639     int matType = home->param->materialType;
640     int bccIndx = home->param->crossSlipBCCIndex;
641
642     if (enabled)
643     {
644         if (matType==MAT_TYPE_BCC)
645         {
646             if (bccIndx==0) { CrossSlipBCC (home); }
647             if (bccIndx==1) { CrossSlipBCCRandom(home); }
648         }
649
650         else if (matType==MAT_TYPE_FCC
651                 ) { CrossSlipFCC(home); }

```

```

652         else if (matType==MAT_TYPE_HCP                      ) { CrossSlipHCP(home); }
653         else if (matType==MAT_TYPE_RHOMBOHEDRAL_VA ) { CrossSlipRhombohedralVa(home); }
654     }
655
656     return;

```

Cross slip is enabled only when the control flag, material, and BCC index agree. These consumers read the same `Param_t` populated earlier; no second control-file parse occurs.

13 A concrete value trace

Control text	Binding/default	Later effect
maxstep=100	Param.cc binds param->maxstep; default is 100.	ParaDiS.cc computes cycleEnd; CLI -maxstep can replace it.
timestepIntegrator="trapezoid"	String destination, default trapezoid.	ParadisStep.cc dispatches to trapezoid integration.
appliedStress=[sixvalues]	V_DBL, count 6.	Force calculation and mobility see the six-component stress tensor.
mobilityLaw="BCC_0"	String destination, default BCC_0.	Mobility table lookup installs material-specific function pointers after broadcast.
remeshRule=2	Integer destination, default 2.	Remesh.cc selects rule-2 remeshing; negative disables it.

14 Restart persistence and debugging

```

56 int WriteCtrl(Home_t *home, char *ctrlFile)
57 {
58     FILE    *fpCtrl;
59
60     if ((fpCtrl = fopen(ctrlFile, "w")) == (FILE *)NULL) {
61         printf("WriteCtrl: Open error %d on %s", errno, ctrlFile);
62         return(errno);
63     }
64
65     /*
66      *   Restart at current cycle count
67      */
68     home->param->cycleStart = home->cycle;
69
70     /*
71      *   Write out all the control parameters
72      */
73     fprintf(fpCtrl, "#####\n");
74     fprintf(fpCtrl, "###          ###\n");
75     fprintf(fpCtrl, "###  ParaDiS control parameter file  ###\n");
76     fprintf(fpCtrl, "###          ###\n");
77     fprintf(fpCtrl, "#####\n\n");
78     WriteParam(home->ctrlParamList, -1, fpCtrl);
79
80     return(fclose(fpCtrl));

```

`WriteCtrl` writes a restart control file by setting the current cycle and calling `WriteParam`.

```

100 void WriteParam(ParamList_t *list, int index, FILE *fp)
101 {
102     int i, j, minIndex, maxIndex, type, count;
103     char *lineFeed, *lBracket, *rBracket;
104
105     /*
106      * We'll be looping over all parameters to be printed. If
107      * a single parameter is selected, set loop indices so we
108      * only iterate once for the specific parameter, otherwise
109      * loop over all parameters.
110      */
111     if (index >= 0) {
112         minIndex = index;
113         maxIndex = index+1;
114     } else {
115         minIndex = 0;
116         maxIndex = list->paramCnt;
117     }
118
119     for (i = minIndex; i < maxIndex; i++) {
120
121         /*
122          * If we're dumping the whole list of parameters, don't
123          * bother dumping out a parameter that is an alias for
124          * for another.
125          */
126         if (((list->varList[i].flags & VFLAG_ALIAS) != 0) && (index < 0)) {
127             continue;
128         }
129
130         /*
131          * Use of some parameters precludes the use of others, in which
132          * case, unused parameters may have been flagged as irrelevant
133          * for this particular simulation. If the current parameter has
134          * has been flagged, there's no need to write it out.
135          *
136          * However, just as a safety net, if value was explicitly supplied
137          * by the user in the restart file, go ahead and write the value
138          * out anyway.
139          */
140         if (((list->varList[i].flags & VFLAG_DISABLED) == VFLAG_DISABLED) &&
141             ((list->varList[i].flags & VFLAG_SET_BY_USER) == 0)) {
142             continue;
143         }
144
145         type = list->varList[i].valType;
146         count = list->varList[i].valCnt;
147
148         if (count > 1) {
149             lineFeed = (char *) "\n";
150             lBracket = (char *) "[";
151             rBracket = (char *) "]";
152         } else {
153             lineFeed = (char *) "";
154             lBracket = (char *) "";
155             rBracket = (char *) "";
156         }
157
158         /*
159          * If the parameter type is not a 'comment', start by

```

```

160  *      writing the identifier/name.
161  */
162      if (type != V_COMMENT) {
163          fprintf(fp, "%s = %s%s", list->varList[i].varName,
164                  lBracket, lineFeed);
165      }
166
167      for (j = 0; j < count; j++) {
168          switch(type) {
169              case V_INT:
170                  fprintf(fp, "  %d%s",
171                          ((int *) (list->varList[i].valList))[j],
172                          lineFeed);
173                  break;
174              case V_DBL:
175                  fprintf(fp, "  %.15e%s",
176                          ((double *) (list->varList[i].valList))[j],
177                          lineFeed);
178                  break;
179              case V_STRING:
180                  if (list->varList[i].valCnt == 1) {
181                      fprintf(fp, "  \"%s\"%s",
182                              (char *) list->varList[i].valList, lineFeed);
183                  } else {
184                      fprintf(fp, "  \"%s\"%s",
185                              ((char **) (list->varList[i].valList))[j],
186                              lineFeed);
187                  }
188                  break;
189              case V_COMMENT:
190                  fprintf(fp, "#\n#  %s\n#",
191                          (char *) list->varList[i].varName);
192                  break;
193
194              } /* end switch */
195
196          } /* for (j = 0; j < count; ...) */
197
198          fprintf(fp, "  %s\n", rBracket);
199
200      } /* for (i = minIndex; i < maxIndex; ...) */
201
202      return;
203  }

```

`WriteParam` skips aliases, suppresses disabled fields unless they were user-set, and prints scalars/vectors according to the same type/count metadata used by the reader. This makes a written control file an auditable snapshot of effective values.

Useful debugger breakpoints are: `Initialize.cc:1789` (before parse), `ReadRestart.cc:94` (lookup), `Parse.cc:324` (conversion), `Initialize.cc:1839` (validation), `Initialize.cc:1909` (broadcast), `Initialize.cc:2004` (mobility mapping), and `ParadisStep.cc:320` (first consumer).

15 Adding a new control parameter

1. Add storage to `Param_t` only if the value is genuinely runtime state.
2. Add one `BindVar` call in `CtrlParamInit` with the exact spelling, destination, type, count, flags, and default.

3. Add cross-field/range checks to `InputSanity` and sentinel derivation to `SetRemainingDefaults` when needed.
4. Consume the field after the MPI broadcast; resolve process-local function pointers after the broadcast.
5. Add a representative control-file test and verify both text and restart output through `WriteParam`.

16 Implementation caveats

- The parser is intentionally small: there is no include directive or recursive file expansion in `ReadControlFile`; it opens one file.
- Unknown keys are tolerated, but unknown multi-value assignments should use brackets so the parser knows where the list ends.
- Numeric lexical validation is weak because conversion uses `atoi/atof`; rely on `InputSanity` for semantic correctness.
- Registry pointers are valid only within a process. Never broadcast `ParamList_t`; broadcast the plain `Param_t` bytes as the implementation does.
- Some `Param_t` members are derived, CLI-only, or internal and are deliberately absent from `CtrlParamInit`; inspect the registry before documenting a key as user-settable.

17 Source index

The line-numbered listings in this document are generated directly from the repository files. The primary source files are `src/Initialize.cc`, `src/Param.cc`, `src/Parse.cc`, `src/ReadRestart.cc`, `src/InputSanity.cc`, `include/Param.h`, `include/Typedefs.h`, and `include/Home.h`; simulation consumers are in `src/ParaDiS.cc`, `src/ParadisStep.cc`, `src/Topology.cc`, `src/Remesh.cc`, and `src/CrossSlip.cc`.